

SAFESCALE

Un système hybride multi-agents et multi-objectifs pour la planification SAFe PI Planning

Optimisation NSGA-II, Critical Chain et HITL conditionnel

Gabriel Leulmi

Mémoire de Master, 2025-2026

Mai 2026

Résumé

La planification d'un *Program Increment* (PI, un cycle de livraison de huit à douze semaines) dans le cadre Scaled Agile Framework [21] est un **problème d'ordonnement sous contraintes de ressources** : on doit décider quoi faire, quand, et par quelle équipe, en respectant les compétences disponibles, les dépendances et les délais business. Ce type de problème (appelé RCPSP dans la recherche opérationnelle [20, 19]) est **non polynomial** : le nombre de plannings possibles explose si vite avec la taille du projet qu'aucun ordinateur ne peut les tester tous. La planification repose donc aujourd'hui sur l'expérience humaine. Le *Release Train Engineer* (RTE, facilitateur du train de livraison) anime le rituel, mais plusieurs autres rôles SAFe interviennent : *Scrum Master*, *Product Owner* (PO), *Chief Product Owner* (CPO), *Engineering Delivery Managers* et les *Development Teams* qui fournissent estimations et *refinement*. Les outils du marché servent à saisir et visualiser ces échanges, pas à explorer plusieurs scénarios candidats ni à les évaluer sur plusieurs critères. Ce mémoire présente **SAFESCALE**, un prototype d'aide à la décision qui assemble trois éléments. D'abord une optimisation par **algorithme génétique multi-objectifs** (NSGA-II [23, 7], qui fait évoluer une population de plannings candidats sur plusieurs générations), étendue à six objectifs dont une **tardivité pondérée par WSJF** (*Weighted Shortest Job First*, le score de priorité économique d'un chantier) : le retard de chaque feature en jours, multiplié par cette valeur business, inspirée d'un cas industriel multi-projets [14], avec une **recherche locale** qui peaufine les meilleurs plannings (approche « mémétique » [3]) et des plannings de départ construits par des heuristiques métier (inspirés d'INSGA-II, pour *Improved NSGA-II* [1, 2]). Ensuite une architecture multi-agents *orchestrée* en cinq étapes, avec un humain dans la boucle de manière **conditionnelle** (sollicité uniquement lorsqu'un problème est détecté) et une trace explicite des décisions. Enfin une **heuristique de marges de sécurité**, inspirée du principe de la *Critical Chain* de Goldratt [24] et de la loi de Kingman [25] : un score de risque déterministe, calculé par une règle et non appris sur des données, insère du temps de réserve après les *stories* les plus risquées. Le système tient compte du ratio durée-effort/durée-calendaire (un story point est rarement exécuté en autant de jours qu'il vaut), de la spécialisation des équipes (*one team owns one feature* [21]) et des goulots de capacité par compétence. Il est instancié sur un jeu de données synthétique reproductible (un ART, neuf équipes, quatre sprints de trois semaines, 112 à 131 *stories* selon le tirage) dont les dimensions sont inspirées d'un entretien avec un Product Owner.

Mots-clés. SAFe ; PI Planning ; RCPSP ; NSGA-II mémétique ; système multi-agents orchestré ; HITL conditionnel ; marges de sécurité inspirées de la *Critical Chain* ; loi de Kingman ; tardivité pondérée WSJF.

1 Introduction

1.1 Contexte

Le *Scaled Agile Framework* (SAFe) définit la planification d'un *Agile Release Train* (ART, voir annexe A) regroupant 5 à 15 équipes et 50 à 125 personnes, organisée autour d'une cérémonie centrale, le **PI Planning**, sur une à deux journées pour planifier les huit à douze semaines suivantes [21].

Quatre termes structurent l'ensemble du document et méritent d'être posés dès à présent. Un **epic** désigne un chantier de grande ampleur, par exemple la refonte d'un module de paiement. Il se décompose en **features**, unités livrables à l'échelle d'un PI. Chaque feature se décompose à son tour en **stories**, unité que l'équipe estime puis réalise. L'effort d'une story s'exprime en **story points**, mesure relative et non en jours. Enfin, le **WSJF** (*Weighted Shortest Job First*) est le score de priorité économique attaché à un chantier : plus il est

élevé, plus le report de sa livraison est coûteux.

Le rituel mobilise plusieurs rôles dont les responsabilités sont précisément distribuées :

- le *Release Train Engineer* (RTE) anime le rituel et arbitre les dépendances entre équipes ;
- le *Chief Product Owner* (CPO) priorise les *features* de son périmètre et les répartit entre équipes ;
- chaque *Product Owner* (PO) porte le contenu de son équipe et présente ses propositions ;
- le *Scrum Master* (SM) facilite la cérémonie côté équipe ;
- les *Engineering Delivery Managers* (EDM) et les autres parties prenantes acceptent ou refusent les engagements proposés ;
- les *Development Teams* produisent les estimations et le *refinement* qui alimentent toutes les décisions.

Aucun de ces rôles ne dispose aujourd'hui d'un outil pour explorer plusieurs scénarios candidats et les éva-

luer quantitativement ; les arbitrages se font à l'œil, sur la base de l'expérience.

1.2 Motivation

Formellement, la tâche du RTE relève de l'ordonancement sous contraintes de ressources : placer des activités dans le temps alors que les personnes sont en nombre limité, que les compétences ne sont pas interchangeables, que certaines activités en attendent d'autres, et que plusieurs projets se disputent les mêmes ressources. Cette classe de problèmes est désignée dans la littérature par *Multi-Skill Resource-Constrained Multi-Project Scheduling Problem* (MS-RCMPSP), extension du RCPSP [20]. Elle est NP-difficile dès une soixantaine d'activités [26, 19] : au-delà de cette taille, aucun algorithme connu ne garantit l'optimum en temps polynomial, et l'on doit se limiter à des solutions approchées obtenues par exploration. « Optimiser » signifie ici trois choses concrètes : (i) générer plusieurs scénarios de PI candidats au lieu d'un seul, (ii) les évaluer simultanément sur plusieurs critères, et (iii) recommander un compromis tout en gardant la décision finale chez l'humain. Cette complexité n'est pas adressée par les outils existants, qui restent centrés sur la saisie et la visualisation des artefacts SAFE. La recherche en optimisation a pourtant beaucoup travaillé sur des problèmes voisins : les variantes hybrides de NSGA-II [3, 4] et NSGA-III [9, 11] donnent de bons résultats sur des instances de RCPSP et d'ordonnement d'atelier. L'objet de ce travail est de transposer ces approches au domaine SAFE, avec ses spécificités propres (WSJF, sprints, équipes spécialisées, statut de *readiness*).

1.3 Contributions

Le mémoire propose six contributions **originales** du système SAFESCALE, chacune transposant ou étendant un résultat publié dans la littérature au contexte spécifique du PI Planning SAFE :

- C1.** Un **sixième critère d'évaluation**, la tardivité pondérée par le WSJF : le retard de chaque feature, multiplié par sa valeur business. Le retard d'un chantier prioritaire pèse ainsi davantage que celui d'un chantier secondaire. Transposition au contexte SAFE d'un objectif validé sur un cas industriel multi-projets [14].
- C2.** Six **plannings de départ** construits par des règles métier, en remplacement d'une population initiale entièrement aléatoire ; l'un d'eux combine valeur business et pression temporelle, adapté de la stratégie INSGA-II [1, 2].
- C3.** Une **phase de raffinement** appliquée au *front 0*, c'est-à-dire à l'ensemble des plannings qu'aucun autre ne surpasse simultanément sur tous les critères : ces plannings sont repris et modifiés par petites retouches ciblées, au lieu d'être seulement recombinaison au hasard. Inspirée du Memetic NSGA-II [3].
- C4.** Une **architecture en cinq étapes** confiées à des

composants spécialisés, où l'intervention humaine n'est requise que lorsqu'un problème est détecté, et non à chaque étape. Dans la lignée des architectures multi-agents couplées à NSGA-II [6].

C5.

Une **marge de sécurité calculée story par story**, proportionnelle à un score de risque déterministe à cinq facteurs, plutôt qu'un coussin uniforme appliqué partout. Heuristique inspirée du principe de Goldratt [24] et de la loi de Kingman [25] ; les cinq coefficients sont fixés par nous et non calibrés sur des données.

C6.

Une **chaîne complète**, du jeu de données au choix final du scénario, rejouable à l'identique sur un dataset synthétique calibré sur un ART réel.

1.4 Plan du mémoire

La section 2 formalise la problématique et précise le **périmètre** du prototype. La section 3 synthétise l'état de l'art en cinq familles. La section 4 décrit le modèle et les paramètres clés. La section 5 présente l'architecture du flow. La section 6 détaille NSGA-II. La section 7 synthétise les décisions de conception et leurs sources. La section 8 recense les problèmes de logique du flow et les corrections apportées. La section 9 décrit le protocole expérimental et les résultats chiffrés de la campagne. La section 10 présente les retours qualitatifs sur la solution. La section 11 expose les limites et ce qui n'est pas pris en compte. La section 12 liste les points à explorer. La section 13 conclut. Deux annexes (A et B) fournissent un glossaire des sigles et des termes techniques.

2 Problématique

2.1 Le processus PI Planning existant

Avant de modéliser le problème, il faut comprendre comment un PI Planning se déroule *aujourd'hui*, sans outil d'optimisation. La description suivante est issue d'un entretien avec un *Product Owner* (PO) opérant sur un ART réel.

Avant le PI Planning. La préparation s'étale sur plusieurs semaines et mobilise plusieurs rôles :

- Le **CPO** (*Chief Product Owner*) **priorise les features** de son périmètre et s'assure qu'elles peuvent atteindre l'état *ready* : il organise pour cela des **ateliers de co-ingénierie** avec les équipes.
- La **durée des epics** est estimée en amont afin de répartir le travail sur les différents PI à venir.
- Le CPO **répartit les features par équipe** ; chaque équipe indique ensuite lesquelles elle est capable d'adresser. Il arrive qu'un epic soit **découpé en features réparties sur plusieurs équipes**.
- Une fois les features allouées, en **pré-PI Planning**, chaque PO prépare le périmètre de son équipe et recense les sujets à traiter.

Pendant le PI Planning. La cérémonie elle-même suit un déroulé bien établi :

- Le **Scrum Master** gère l'animation, le **PO** apporte la matière des discussions.
- On **liste les features retenues** pour le PI.

- Chaque équipe réalise un **poker planning** après le découpage des features en *stories*, en visant une taille de **8 SP** (story points) **maximum par story**.
- Les équipes **proposent les features** qu'elles se déclarent capables d'adresser. C'est à ce stade que le statut de *readiness* entre en jeu : si une équipe identifie des **incertitudes** sur certains items à développer (spécifications floues, dépendance externe non confirmée, etc.), elle ne s'**engage** pas dessus pour le PI à venir (dans le vocabulaire SAFe, elle ne se *commit* pas : elle refuse de prendre l'engagement de livrer, faute de visibilité suffisante), et peut proposer de les **repousser au PI suivant**.

Pendant l'exécution. La planification reste **volontairement générale** : à chaque sprint, les équipes tiennent un **meeting de réajustement** pour corriger le plan en fonction de la réalité du terrain.

Limites de ce processus. Ce déroulé est mature et éprouvé, mais il reste entièrement **manuel** : la répartition features→équipes, la détection des surcharges et des conflits de dépendances, l'arbitrage entre priorités concurrentes reposent sur l'expérience humaine. Aucune **exploration algorithmique** de scénarios alternatifs n'est possible, et aucune **évaluation quantitative multi-critères** n'assiste la décision. C'est précisément cet écart que SAFESCALE cherche à combler, en s'insérant comme une aide à la décision **sans modifier les rôles existants** (CPO, PO, Scrum Master, RTE).

2.2 Ce qui rend ce problème difficile

Le PI Planning cumule cinq caractéristiques qui, prises ensemble, placent le problème hors de portée d'une résolution exacte [20, 19] :

- plusieurs chantiers se partagent les mêmes personnes, et ce qu'on accorde à l'un est retiré à l'autre (*multi-projet*) [16] ;
- les personnes ne sont pas interchangeables : chacune ne maîtrise qu'un sous-ensemble de compétences, et certaines compétences ne sont détenues que par une poignée d'individus (*multi-compétences*) [15] ;
- six critères doivent être satisfaits simultanément alors qu'ils se contredisent : durée totale, risque de retard, marge de sécurité, surcharge des personnes, dépassement de capacité d'une équipe sur un sprint, et retard pondéré par la valeur business (*multi-objectifs*, détaillés section 6) [7, 19] ;
- une feature est portée par une seule équipe, principe que SAFe formule par « *one team owns one feature* » [21] ;
- une dépendance déclarée entre deux features contraint l'ensemble de leurs stories, et non les seules stories concernées [14].

2.3 Cinq familles de conflits récurrents

L'entretien mené avec un Product Owner, complété par notre modélisation, fait apparaître cinq familles récurrentes de conflits. Leur résolution manuelle, qui mobilise l'ensemble des rôles SAFe (RTE, CPO, PO, Scrum Master, EDM et équipes en dépendance), est ce

que l'on cherche à instrumenter :

1. **Dépassement de capacité** : une équipe se voit attribuer plus de travail qu'elle ne peut en absorber (*CAPACITY_VIOLATION*).
2. **Dépendance violée** : une story est programmée avant celle dont elle dépend (*DEPENDENCY_BREACH*).
3. **Débordement de périmètre** : des features que l'ART ne peut pas absorber dans le PI courant et qui doivent être reportées (*FEATURE_OVERFLOW*).
4. **Échéance menacée** : une feature se termine après la date qui lui était fixée (*DEADLINE_RISK*).
5. **Charge déséquilibrée** : plus de 30 % d'écart de charge entre deux équipes (*LOAD_IMBALANCE*).

Ces cinq conflits sont *interdépendants* : résoudre une surcharge crée souvent une violation de dépendance, qui crée un risque de *deadline* [14].

2.4 Question de recherche

Peut-on concevoir un système hybride combinant optimisation multi-objectifs, architecture multi-agents et modèle de risque, pour automatiser la génération et l'évaluation de scénarios de PI Planning SAFe, tout en conservant à l'humain un rôle décisionnel effectif ?

2.5 Périmètre du prototype

Le prototype se concentre sur un cas d'étude défini précisément, afin de garantir la *reproductibilité* et la lisibilité des résultats :

- **Un seul ART**, neuf équipes spécialisées (*backend, frontend, data, devops, mobile, qa, fullstack fall-back*), conformément à la fourchette 5-15 équipes par ART [21].
- **Un PI de quatre sprints de trois semaines** (60 jours ouvrés), format relevé lors de l'entretien PO et cohérent avec la fourchette 8-12 semaines [21].
- **Une cinquantaine de ressources** (5-7 par équipe, 52 sur le jeu de référence), 9 epics, ≈2 features par epic (≈18 features), 5 à 10 *stories* par feature, soit 112 à 131 *stories* selon le tirage (section 9). La *story* est l'unité de planification : il n'y a pas de niveau *tâche*, conformément au processus réel observé (section 2.1).
- **Conversion effort→durée** : la valeur empirique varie d'une équipe à l'autre, typiquement entre **1 et 2 jours ouvrés par story point**. On utilise **1 SP = 1,7 jour** comme valeur pivot, en autorisant un tirage par équipe dans l'intervalle [1,0 ; 2,0].
- **Données synthétiques** générées avec une *seed* fixe (*seed=42*), garantissant la reproductibilité intégrale des expérimentations sans accès à des données réelles confidentielles.
- **Mode batch** : on planifie l'intégralité d'un PI en une session de bout en bout, déclenchée par le RTE. « Batch » ne signifie pas « sans interaction » : les cinq étapes du flow comportent des points HITL conditionnels où le RTE intervient en cours d'exécution (cf. section 5). C'est le *replanning en flux continu* entre deux PI (par exemple en réaction à un événement extérieur) qui est hors périmètre.

Ce périmètre fixe le cadre du travail. Les éléments qui en sortent (données réelles Jira, multi-ART simultané, *replanning* temps réel, apprentissage par renforcement) sont explicitement reportés en limites (section 11) et

en perspectives (section 12).

3 État de l'art

L'état de l'art est organisé en cinq familles couvrant les axes mobilisés par le système : algorithmes évolutifs multi-objectifs, extension à plus de trois objectifs, RCPSP hybride et multi-projets, pratiques officielles SAFe, et intégration du machine learning dans les MOEA.

3.1 Famille 1, NSGA-II et ses variantes

NSGA-II [23] reste, vingt ans après sa publication, « de loin la *métaheuristique* la plus populaire pour les problèmes d'optimisation multi-objectifs » [7]. Trois axes d'amélioration récents structurent notre contribution :

Axe 1, Initialisation ciblée. *INSGA-II* [1] démontre empiriquement sur 24 instances qu'une initialisation aléatoire dégrade fortement la convergence ; les stratégies adossées aux objectifs (WSJF, deadline-first, chemin critique, load-balance) améliorent significativement le taux d'optimisation [1, 2]. Notre seed C2 « WSJF / deadline urgency » prolonge cette idée en combinant valeur business et pression temporelle.

Axe 2, Hybridation mémétique. NSGA-II progresse par « grands sauts » aléatoires (croisements et mutations) : il explore très large, mais ne *peaufine* jamais une solution. Il peut passer juste à côté d'un optimum sans s'y arrêter. L'idée mémétique consiste à ajouter à NSGA-II une seconde mécanique complémentaire, la *recherche locale greedy* : on prend les meilleures solutions du moment et on leur applique des **petites retouches ciblées** (déplacer une story d'une équipe à une autre, inverser l'ordre de deux stories), en ne gardant que celles qui améliorent un objectif. Le tableau ci-dessous illustre la différence :

Sans recherche locale	Avec (mémétique)
On explore en grand au hasard, mais on ne polit aucune solution.	On explore en grand au hasard, puis on polit les meilleures par petites modifications.
Bonne couverture, qualité fine moyenne.	Bonne couverture <i>et</i> qualité fine.

Memetic NSGA-II [3] applique cette recette aux meilleurs individus du front 0 ; sur des problèmes d'ordonnancement d'atelier (*flexible job shop*), les auteurs obtiennent des fronts de qualité supérieure à ceux d'un solveur exact, pour un temps de calcul bien moindre. C'est la base de notre contribution C3.

Axe 3, NSGA-II + Q-Learning. Dans NSGA-II classique, le **moment** où chaque tâche démarre est choisi par des règles *fixes* (« le plus tôt possible une fois les dépendances satisfaites », par exemple). L'idée de cet axe est de remplacer cette règle figée par une **politique apprise** : un agent d'*apprentissage par renforcement* essaie différentes dates de début sur de nom-

breux plannings, reçoit un retour « c'était une bonne ou une mauvaise idée » selon la qualité du planning final, et *apprend* progressivement quelles dates marchent le mieux. [4] formalise cela comme un *processus de décision markovien*, résolu par l'algorithme *Q-Learning*. Le tableau ci-dessous illustre la différence :

Sans Q-Learning	Q-Learning	Avec (NSGA-II + Q-Learning)
Dates de début choisies par une règle fixe, identique sur tous les plannings.	Dates de début choisies par une politique <i>apprise</i> sur des centaines de plannings antérieurs.	
Pas besoin d'entraînement, mais qualité limitée par la règle.	Coûteux à entraîner, mais meilleure qualité finale.	

Les résultats publiés surpassent NSGA-II seul, ainsi que deux autres familles d'algorithmes inspirés du vivant : l'optimisation par essaim particulaire (PSO) et les colonies de fourmis (ACO). L'entraînement est coûteux mais l'approche est riche en information ; on l'identifie comme perspective P3.

Ce qu'on retient et ce qui manque : ces trois axes valident séparément l'initialisation ciblée, la recherche locale et l'apprentissage du timing, mais aucun de ces travaux ne traite un problème structuré comme SAFe (hiérarchie epic/feature/story, contrainte *one team owns one feature*, WSJF). Ils restent sur des bancs d'essai génériques de *job shop* ; la transposition au PI Planning n'est pas immédiate et constitue justement une partie de notre travail.

3.2 Famille 2, NSGA-III pour 5+ objectifs

Avec six objectifs, la *crowding distance* de NSGA-II devient moins efficace pour préserver l'uniformité du front [9]. NSGA-III remplace ce mécanisme par un système de **points de référence** [11, 10], et obtient de meilleurs fronts que NSGA-II sur les benchmarks RCPSP de [11].

Ce qu'on retient et ce qui manque : NSGA-III est clairement plus adapté à notre nombre d'objectifs, et c'est pourquoi il figure en perspective P2. Mais il introduit un paramétrage supplémentaire, le placement des points de référence, qui demande une étude à part entière. Commencer par NSGA-II, mieux documenté et plus simple à déboguer, est un choix assumé de prototype, pas une fin en soi.

3.3 Famille 3, RCPSP hybride et multi-projets

Le *Hybrid Genetic Algorithm* de référence [12] reste une base classique : c'est un *algorithme génétique* standard auquel on greffe un *repair-based decoding* (cf. section 6) pour rendre la *mutation* toujours valide. [13] introduit le concept de *dense genes*, ce sont les gènes qui « saturent » l'utilisation d'une ressource (proches du chemin critique). L'idée est d'utiliser ces gènes denses comme *points d'ancrage* pendant le *croisement*, plutôt que de mélanger les chromosomes au hasard : les bonnes parties d'une solution sont préservées. Sur les cas multi-projets industriels, [14] a inspiré notre 6^e objectif : la tardivité pondérée par WSJF (transpo-

sition de la *weighted tardiness* pondérée par criticité business). Pour la durabilité, [16] adapte NSGA-II au multi-projet, et [15] traite le cas des profils de ressource flexibles, où la quantité de personnes affectée à une activité peut être ajustée en cours d'exécution, au moyen d'un algorithme d'essaim multi-objectifs. Pour l'incertitude des durées, [17] convertit l'incertain (optimiste/pessimiste/probable) en objectifs; [18] modélise les coûts variables dans le temps. La revue [19] conclut que l'hybridation domine généralement les algorithmes purs.

Ce qu'on retient et ce qui manque : cette famille fournit les briques techniques (*décodage*, croisement, objectif de tardivité) mais raisonne en « activités » abstraites. Elle ignore deux réalités du PI Planning : le découpage en sprints à capacité fixe, et le rôle d'un décideur humain qui tranche en fin de processus. Notre travail réutilise les briques mais les recompose autour de ces deux contraintes.

3.4 Famille 4, SAFe officiel et flow lean

La source officielle SAFe [21] fournit les paramètres normatifs : 5-15 équipes par ART, 50-125 personnes, 8-12 semaines par PI, ROAM board pour les risques. Reinertsen 2009 [25] formalise la relation entre utilisation et *lead-time* (loi de Kingman) : au-delà de 80 % d'utilisation, le lead-time devient super-linéaire. C'est la base de notre buffer 80 % bloquant (DEC-016) et de la valeur pivot 2.0 du *span_factor*.

Ce qu'on retient et ce qui manque : SAFe décrit *quoi* faire (les cérémonies, les rôles, les artefacts) mais pas *comment* optimiser le contenu d'un PI. La documentation officielle reste volontairement méthodologique et n'offre aucun outil quantitatif. C'est précisément l'espace que ce mémoire cherche à occuper.

3.5 Famille 5, ML + algorithmes évolutifs

Cette famille combine **apprentissage automatique** (ML) et **algorithmes évolutifs multi-objectifs** (MOEA), la catégorie dont NSGA-II fait partie. L'idée commune : utiliser un modèle ML, appris sur l'historique des projets, pour orienter l'optimisation vers des solutions à *risque* prédit plus faible. Le *survey* [22] en donne un panorama. *ENSGA* (*Enhanced NSGA-II, idem*) est un exemple : il glisse un modèle ML qui prédit la fatigue ou le risque des opérateurs dans la fonction objectif, et optimise à la fois la durée totale (*makespan*) et ces critères humains. Le multi-agent NSGA-II [6] traite quant à lui les problèmes à beaucoup d'objectifs en *répartissant* le calcul de la *crowding distance* (cf. section 6) entre plusieurs agents pour mieux couvrir le front.

Ce qu'on retient et ce qui manque : ces travaux montrent qu'un modèle de risque peut entrer dans la fonction objectif d'un MOEA. En revanche, ils mobilisent souvent du ML appris sur des historiques volumineux. Faute d'un tel historique, on a délibérément choisi un score de risque *déterministe* (équation 7), moins ambitieux mais reproductible et explicable, un compromis assumé.

3.6 Synthèse du positionnement

Prises séparément, ces cinq familles couvrent chacune un aspect du problème; aucune ne les réunit pour le PI Planning SAFe. C'est cette combinaison qui constitue l'apport du mémoire, résumé table 1 : (i) NSGA-II multi-objectifs adapté SAFe avec mémétique et seed *urgency*, (ii) MAS orchestré avec HITL conditionnel et trace, (iii) marges de sécurité locales, inspirées de la Critical Chain, au niveau *story*.

Dimension	Existant	SAFESCALE
Optim.	RCPSP mono-obj.	NSGA-II 6-obj. + LS
MAS	BDI / centralisé	Orchestré + HITL
Risque	Absent	Marges ciblées + score
Domaine	Ordonn. générique	SAFe (WSJF, sprints)
HITL	Final seul	Conditionnel par étape

TABLE 1 – Positionnement comparatif de SAFESCALE.

4 Modèle et données

4.1 Hiérarchie SAFe modélisée

SAFe distingue deux axes qu'il faut éviter de confondre : un axe d'**organisation** (qui travaille avec qui) et un axe de **décomposition du travail** (qui est découpé en quoi). Notre modèle reflète les deux séparément :

organisation : $\text{ART} \rightarrow \text{Team} \rightarrow \text{Ressource}$

décomposition : $(\text{Initiative}) \rightarrow \text{Epic} \rightarrow \text{Feature} \rightarrow \text{Story}$

Le niveau *Initiative* (entre parenthèses) existe en SAFe au-dessus de l'epic comme grand thème stratégique ; on le mentionne pour la complétude mais on ne le modélise pas, faute d'effet sur la planification d'un seul PI. Notre modèle s'arrête à la *story* sur l'axe de décomposition, l'unité réellement estimée et planifiée dans le processus observé (section 2.1).

S'y ajoutent les compétences (*Skill*), les dépendances de *features* et de *stories*, et les artefacts de planification (*PIPlanning*, *StorySchedule*, *Scenario*, *DecisionTrace*). Conformément au processus réel, il n'y a **pas de niveau tâche** : la *story* est le grain le plus fin, estimée en *story points* (SP) lors du poker planning, avec un plafond de 8 SP par story.

4.2 Conversion effort, durée et étalement

L'effort d'une *story* est exprimé en *story points*. En pratique, un story point représente entre 1 et 2 jours-effort selon l'équipe; on retient une valeur pivot de 1,7 jour/SP, modulable par équipe pour refléter cette variabilité :

$$d_s = SP_s \times \alpha_{\text{team}(s)}, \quad \alpha_{\text{team}} \in [1,0; 2,0], \quad \alpha_{\text{défaut}} = 1,7 \quad (1)$$

Mais une story estimée à d_s jours-effort n'est jamais exécutée sur d_s jours calendaires consécutifs : réunions récurrentes, *context-switching*, blocages tech-

niques étalent le travail. On modélise cela par un **facteur d'étalement** :

$$calendar(s) = \lfloor d_s \cdot \text{span_factor}_{\text{team}(s)} \rfloor \quad (2)$$

Le `span_factor` a une valeur pivot de 2,0, mais il n'est pas identique pour toutes les équipes : le générateur tire pour chaque équipe une valeur dans l'intervalle $[1,8; 2,2]$. Cette valeur pivot traduit un résultat de la théorie des files d'attente, connu sous le nom de loi de Kingman [25] : une équipe occupée à environ la moitié de son temps sur un sujet met à peu près deux fois plus de temps calendaire que le temps de travail effectif. Une story demandant cinq jours de travail occupe donc une dizaine de jours au calendrier, le reste étant absorbé par les réunions, les attentes et les changements de contexte. L'écart $\pm 0,2$ traduit le fait qu'une équipe expérimentée subit moins d'étalement (borne basse) qu'une équipe junior (borne haute).

4.3 Spécialisation des équipes

Chaque équipe a une spécialisation parmi sept catégories (*backend*, *frontend*, *data*, *devops*, *mobile*, *qa*, *full-stack*), conforme à la notion SAFe d'*Agile Team* [21]. La distribution des compétences suit une règle 80/20 : 80 % des skills d'une ressource viennent du cœur de spécialité de son équipe, 20 % peuvent déborder. Les compétences rares (*data*, *mobile*, *devops*) sont *exclusives* aux équipes correspondantes, ce qui doit permettre de provoquer un goulot capacité observable (cf. section 11).

4.4 Dataset synthétique de référence

Le générateur produit, pour `seed=42`, un dataset reproductible aux dimensions relevées sur un ART réel (section 2.1) :

Entité	Quantité
ART	1
Équipes spécialisées (5-15 SAFe)	9
Ressources (5-7 / équipe)	52
Epics	9
Features (≈ 2 / epic)	≈ 18
Stories (5-10 / feature, ≈ 8 SP)	112-131
PI : 4 sprints $\times$ 3 semaines	60 j

5 Architecture et flow

5.1 Vue technique en trois couches

L'application est structurée en **frontend React + TypeScript** (Tailwind, Recharts), **backend Python + Django + DRF** (*Django REST Framework*), et **base PostgreSQL** (figure 1). Un orchestrateur central (`discussion_flow.py`) pilote des agents spécialisés via un protocole synchrone requête/réponse, à la manière du multi-agent NSGA-II proposé par [6]. Les LLM interviennent exclusivement comme **traducteurs** (*rationale* en langage naturel), jamais comme décideurs ; ils sont *mockables* pour la reproductibilité des tests.

5.2 Flow en cinq étapes

Le flow est résumé figure 2 :

1. **Préparation** (*readiness*) : classement de chaque feature en READY / BORDERLINE / NOT_READY, conforme au principe de « backlog refinement » [21].

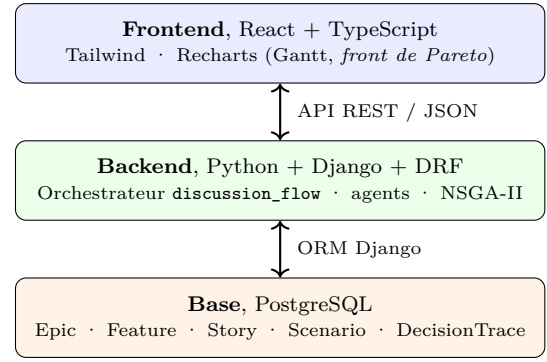


FIGURE 1 – Architecture en trois couches de SAFESCALE.

Pour chaque équipe, le *Product Owner* reste le propriétaire du backlog ; il s'appuie sur de nombreux autres acteurs (CPO, EDM, Scrum Master, équipes en dépendance) qui influencent les priorités et l'état de préparation. HITL si des features restent non prêtes.

2. **Dépendances** : on construit le graphe orienté des dépendances entre features et entre stories (le DAG), puis on le parcourt en **profondeur d'abord** (*Depth-First Search*, DFS) pour deux usages : (i) détecter les cycles de dépendance, qui rendraient la planification impossible ; (ii) calculer le *chemin critique*, la plus longue chaîne de dépendances qui fixe la durée minimale du PI [14].
3. **Capacité & allocation** : chaque équipe construit son plan capacitaire. **HITL bloquant si une équipe dépasse 80 % de sa capacité** [25].
4. **Planification** : NSGA-II (encodage *story* → ressource) produit un front de Pareto, dont on extrait $K = 5$ scénarios **diversifiés**. La diversification est évaluée par la *distance de Hamming* sur l'assignation feature → équipe : deux scénarios sont d'autant plus différents qu'ils affectent plus de features à des équipes distinctes. La sélection *farthest-first* (cf. contribution 6) garantit que chaque nouveau scénario présenté est le plus éloigné possible des précédents, ce qui évite de proposer cinq variantes du même compromis.
5. **Choix du scénario : décision RTE explicite**, inspirée du *max-regret* de [4] et du ROAM board SAFe [21].

5.3 HITL conditionnel

Plutôt que de demander à chaque étape « veux-tu continuer ? », chaque agent décide localement s'il peut trancher seul ou s'il doit ouvrir un point HITL. Cette logique « conditionnelle » est ce qui différencie SAFESCALE de la plupart des outils SAFe existants où l'humain est soit absent, soit sollicité partout.

5.4 Traçabilité (DecisionTrace)

Chaque décision est persistée dans un modèle *DecisionTrace* (question posée, décision prise, justification, alternatives écartées, *snapshot* des inputs, diff des outputs, directives humaines, score de confiance, *prompt* et réponse LLM). La traçabilité multi-niveaux est nécessaire pour la défense soutenue dans la revue

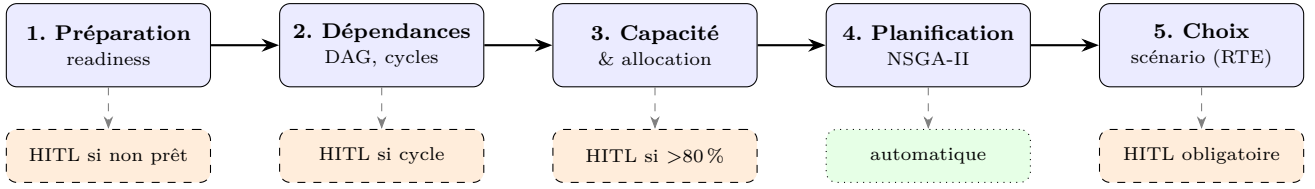


FIGURE 2 – Le flow en cinq étapes. L'étape 4 (NSGA-II) est entièrement automatique ; les étapes 1, 2 et 3 ouvrent un point HITL *conditionnel*, seulement si un problème est détecté, tandis que l'étape 5 requiert toujours une décision humaine explicite.

Comprehensive Review sur NSGA-II combinatoire [7].

6 NSGA-II : formulation et implémentation

6.1 Vocabulaire des algorithmes évolutionnaires

NSGA-II appartient à la famille des algorithmes évolutionnaires, qui procèdent par analogie avec la sélection naturelle. Une solution candidate, ici un planning complet, y est appelée *individu* ; l'ensemble des solutions examinées simultanément forme la *population* ; la description interne d'un individu est son *chromosome*, découpé en *gènes*. Une génération consiste à combiner deux plannings existants (*croisement*), à en modifier légèrement quelques-uns (*mutation*), puis à ne conserver que les meilleurs. Répété sur une centaine de générations, ce cycle fait progresser la population sans qu'aucune solution ait été construite directement.

6.2 Encodage du chromosome

Un individu est une liste $\mathbf{x} = (x_1, \dots, x_n)$ où $x_i = (story_id_i, resource_id_i)$: chaque gène affecte une *story* à une ressource. L'ordre des gènes encode l'ordre d'exécution. La validité topologique est garantie au croisement et à la mutation par un *repair-based decoding*, un mécanisme qui, après chaque opération génétique, parcourt le chromosome et réordonne automatiquement les gènes en conflit (dépendance violée, ressource déjà occupée) au lieu de rejeter la solution, hérité de [12].

6.3 Décodage

Le décodage transforme un chromosome en planning daté. Chaque story démarre au plus tôt, c'est-à-dire dès que ses prédécesseurs sont achevés et que la ressource qui lui est affectée redevient disponible. On lui ajoute sa durée calendaire, puis sa marge de sécurité, après quoi la ressource est de nouveau libre :

$$earliest_i = \max \{ end_p + buffer_p \mid p \prec i \} \quad (3)$$

$$start_i = \max (earliest_i, avail_{r(i)}) \quad (4)$$

$$end_i = start_i + calendar(s_i) \quad (5)$$

$$avail_{r(i)} \leftarrow end_i + buffer_i \quad (6)$$

où $calendar(s_i)$ est la durée calendaire de la *story* s_i (équations 1–2) et $buffer_i$ la marge de sécurité définie ci-après. La marge apparaît deux fois, et c'est voulu : le successeur attend la fin de son prédécesseur **plus** la marge de celui-ci, et la ressource ne se libère qu'après cette même marge. Sans cela, la marge existerait sur le papier sans jamais protéger personne.

6.4 Marge de sécurité inspirée de la Critical Chain

La *Critical Chain* de Goldratt [24] repose sur un principe simple : plutôt que gonfler chaque estimation « au cas où », marges qui se dissipent toujours, on retire les marges individuelles et on les regroupe là où se concentre la variance ; le principe est repris et formalisé en logique floue par [17]. Nous en reprenons ce principe, et lui seul.

Deux réserves doivent être posées d'emblée. La méthode de Goldratt prévoit aussi un **tampon global** en fin de projet, que SAFESCALE n'implémente pas (section 11) : nos marges sont exclusivement locales, story par story. Et la façon dont nous estimons la variance d'une story, ci-dessous, est une règle que nous avons écrite, avec des coefficients que nous avons fixés. Il s'agit donc d'une **heuristique inspirée de la Critical Chain**, non d'une implémentation de la méthode.

Cette règle attribue à chaque *story* un score de risque déterministe :

$$risk(s) = 0,30 \cdot \tilde{c} + 0,20 \cdot \tilde{\sigma} + 0,20 \cdot \tilde{p} + 0,20 \cdot \mathcal{K}_{rare}(s) + 0,10 \cdot \tilde{d} \quad (7)$$

$$buffer(s) = \text{round}(risk(s) \cdot 5)$$

où $\tilde{c}, \tilde{\sigma}, \tilde{p}, \tilde{d}$ sont respectivement la complexité, le **span_factor**, le nombre de prédécesseurs et l'effort (en SP), normalisés ; $\mathcal{K}_{rare}$ vaut 1 si la compétence requise est *data*, *mobile* ou *devops* (catégories sous-représentées). Cette modélisation prolonge l'idée de [22], qui valide empiriquement l'intégration d'un modèle de risque dans la fonction objectif d'un MOEA, et celle de [17], qui convertit l'incertitude en objectifs.

6.5 Six objectifs (à minimiser)

Ces six objectifs **ne sont pas indépendants** : au contraire, ils se contredisent. Si on essaie de compresser le *makespan*, on parallélise plus, donc on surcharge les ressources et le risque monte. Si à l'inverse on garde de la marge pour baisser le risque, on allonge le *makespan* et certaines features peuvent finir après leur deadline. Il n'existe donc pas un seul planning qui soit le meilleur sur les six critères en même temps. C'est exactement pour ça qu'on utilise un *algorithme évolutif* multi-objectifs : au lieu de viser un optimum unique, NSGA-II construit un *front de Pareto*, c'est-à-dire un ensemble de compromis où chaque gain sur un critère se paie ailleurs.

Pondération. Pendant la recherche, NSGA-II **ne pondère pas** les objectifs : tous sont mis sur un pied

d'égalité via la relation de dominance (équation 9). La pondération arrive uniquement à la toute fin, quand il faut choisir un scénario « recommandé » parmi ceux du front. À ce moment-là, on calcule un *score composite* (équation 13) dont les poids w_j peuvent être ajustés par le RTE. Par défaut, les six poids sont ceux du vecteur (0,22 ; 0,18 ; 0,15 ; 0,13 ; 0,15 ; 0,17), dans l'ordre des six objectifs énumérés ci-dessous.

Les six objectifs sont :

1. Makespan $C_{\max}$ [23].
2. Score de risque moyen $\frac{1}{n} \sum_i risk(s_i)$, calculé par la règle déterministe de l'équation 7 et non par un modèle appris [22].
3. Buffer déficit (marge manquante).
4. Surcharge ressource (parallélisme).
5. Violation de capacité par équipe et sprint [21].
6. **Tardivité pondérée WSJF** (contribution C1) :

$$f_6 = \frac{1}{n} \sum_i WSJF_i \cdot \max(0, end_i - deadline_i) \quad (8)$$

Transposition au contexte SAFe de l'objectif *weighted tardiness* validé sur un cas industriel multi-projets [14].

6.6 Dominance et front de Pareto

Dominance de Pareto stricte [23] :

$$\mathbf{x} \prec \mathbf{y} \iff \forall j, f_j(\mathbf{x}) \leq f_j(\mathbf{y}) \wedge \exists k, f_k(\mathbf{x}) < f_k(\mathbf{y}) \quad (9)$$

Lorsque plusieurs solutions sont équivalentes au sens de cette relation, il faut néanmoins en écarter certaines pour maintenir la taille de la population. NSGA-II conserve alors en priorité les plus **isolées**, celles qui n'ont pas de voisin proche dans l'espace des objectifs, afin de préserver la diversité du front. La *crowding distance* mesure cet isolement en sommant, sur chaque objectif, l'écart entre les deux voisins immédiats de la solution [23] :

$$d_c(i) = \sum_{j=1}^M \frac{f_j(i+1) - f_j(i-1)}{f_j^{\max} - f_j^{\min}} \quad (10)$$

6.7 Six seeds intelligents

NSGA-II démarre avec une **population initiale** de plannings candidats, typiquement 40 à 60 individus. Si on génère ces plannings totalement au hasard, l'algorithme passe les premières générations à « rattraper son retard ». INSGA-II [1, 2] a montré qu'il est bien plus efficace d'**injecter quelques plannings construits par des heuristiques simples** dans la population initiale : l'évolution démarre alors près de « bonnes régions » de l'espace de recherche.

Nos six heuristiques de génération produisent chacune un planning initial complet en ordonnant les stories selon un critère métier : (1) WSJF descendant, valeur business d'abord ; (2) deadline ascendante, urgence d'abord ; (3) load-balancé, répartir la charge équitablement ; (4) chemin critique d'abord [14] ; (5) score de risque minimal d'abord [22] ; (6) **ratio WSJF / deadline urgency** (C2) qui combine valeur et pression temporelle :

$$urgency(f) = \frac{WSJF_f}{\max(1, deadline_f - now)} \quad (11)$$

Le reste de la population initiale est complété par des plannings aléatoires pour garder de la diversité.

6.8 Recherche locale mémétique (C3)

Toutes les 5 générations, recherche locale *greedy* sur les 15 % supérieurs du front 0 (réassignation de ressource, swap d'ordre), à la suite de [3]. On ne garde la mutation que si elle ne dégrade aucun objectif (test de non-dominance).

6.9 Sélection des K=5 scénarios diversifiés

Le front de Pareto contient souvent des dizaines de solutions non-dominées. On ne peut pas toutes les présenter au RTE, et choisir les cinq « meilleures » ne marche pas : on tombe vite sur cinq plannings qui se ressemblent tous, parce qu'ils sont voisins du même optimum. La revue [7] et le multi-agent NSGA-II [6] insistent sur l'importance d'une **bonne couverture** du front, pas seulement sur la qualité brute des solutions retenues.

Mesurer la différence entre deux plannings. On utilise la *distance de Hamming*, qui compte le nombre de positions où deux plannings diffèrent. Ici, chaque « position » correspond à une feature, et la valeur stockée est « quelle équipe l'a prise ». Deux plannings où toutes les features sont allouées aux mêmes équipes ont une distance de 0 ; deux plannings qui répartissent chaque feature à une équipe différente sont à distance maximale.

Choisir les 5 plannings les plus différents. On part du scénario recommandé par le score composite, puis on ajoute un à un les quatre suivants en appliquant une règle dite *farthest-first* : à chaque étape, on prend dans le front celui qui est le **plus loin** des scénarios déjà retenus. Cette règle s'écrit :

$$\mathbf{x}_{k+1} = \arg \max_{\mathbf{x} \in \mathcal{F}^0 \setminus \mathcal{S}_k} \min_{\mathbf{y} \in \mathcal{S}_k} d_H(\mathbf{x}, \mathbf{y}) \quad (12)$$

$\mathcal{F}^0$ est l'ensemble du front Pareto, $\mathcal{S}_k$ les k scénarios déjà choisis, et d_H la distance de Hamming. À chaque ajout, on garantit que le nouveau scénario est différent de tous les précédents, ce qui évite les cinq variantes du même compromis.

Score composite (recommandé). Pour désigner le scénario « recommandé » par défaut, on normalise chaque objectif entre 0 et 1 et on en fait une moyenne pondérée :

$$score(\mathbf{x}) = \sum_{j=1}^6 w_j \cdot \frac{f_j(\mathbf{x}) - f_j^{\min}}{f_j^{\max} - f_j^{\min}} \quad (13)$$

$f_j^{\min}$ et $f_j^{\max}$ sont les bornes observées sur le front ; les w_j sont les poids des six objectifs. Ils valent par défaut (0,22 ; 0,18 ; 0,15 ; 0,13 ; 0,15 ; 0,17) et somment à 1 : la durée pèse le plus, la surcharge des personnes le moins. Ces valeurs ont été fixées par nos soins et n'ont pas été validées expérimentalement ; elles sont ajustables par le RTE et figurent parmi les choix assumés (section 9.9).

6.10 Conseil de retrait au step 3

Quand une équipe dépasse 80 % de charge [25], on calcule un score d'impact déterministe :

$$\text{impact}(f) = \text{WSJF}_f \cdot (1 + \text{downstream_blocks}_f) \quad (14)$$

Le score mesure ce que l'on perd en retirant la *feature* : sa valeur business, multipliée par le nombre de *features* qu'elle débloque. Le 1+ évite d'annuler le produit pour une *feature* sans successeur. Plus le score est bas, plus la *feature* est candidate au retrait : faible valeur business et peu de successeurs bloqués derrière elle. La cascade applique `on_delete=CASCADE` au niveau Feature→Story et signale les successeurs DAG potentiellement orphelins.

6.11 Détection de goulot capacité

$$\text{demand}(s) = \sum_{t: \text{skill}(t)=s} d_t \quad (15)$$

$$\text{supply}(s) = \sum_{r: s \in \text{skills}(r)} \text{avail}_r \cdot \text{PI}_{\text{days}} \quad (16)$$

$$\text{gap}(s) = \max(0, \text{demand}(s) - \text{supply}(s)) \quad (17)$$

7 Décisions de conception

Le tableau 2 synthétise les décisions de conception majeures avec leur justification. Deux sources ont nourri ces choix : la littérature scientifique récente d'une part, et d'autre part un **entretien semi-directif avec un Product Owner** opérant sur un ART SAFe. Cet entretien n'a pas valeur d'étude de terrain exhaustive, mais il a permis de cadrer des éléments qu'aucun article ne documente précisément, le format de PI à 4 × 3 semaines, la conversion empirique story points/jours, ou l'application stricte du buffer 20 %. Quand une décision s'appuie sur cet entretien, la mention *Entretien PO* l'indique dans le tableau.

Une colonne **Statut** indique ce que chaque décision vaut aujourd'hui, et il faut la lire sans indulgence :

- **Mesuré** : la campagne expérimentale (section 9) montre chiffres à l'appui que la décision améliore le résultat ;
- **Vérifié** : un test automatisé garantit que le système se comporte comme la décision le prévoit, sans que cela dise si c'était le bon choix ;
- **Assumé** : le choix repose sur la littérature ou sur l'entretien, mais rien dans ce mémoire ne le démontre. Ces cas sont repris et discutés en section 9.9.

8 Problèmes de logique du flow et solutions

Le flow décrit en section 5 n'a pas été conçu d'un seul jet. Sa première version était plus naïve, et plusieurs défauts de logique ne sont apparus qu'en faisant tourner le système de bout en bout sur le dataset de référence : un consensus qui se forme sur des chiffres trompeurs, une étape qu'on traverse sans s'en rendre compte, un optimiseur qui produit un plan absurde mais « valide ». Cette section rassemble ces défauts et la correction que j'ai apportée à chacun. Ce ne sont pas des bugs de code mais des erreurs de **conception du flow** ; je les

présente sous la forme *symptôme* puis *correction*, en renvoyant à la décision DEC-XXX correspondante.

8.1 Agents équipes négociant sur des agrégats SP

Symptôme. La première version du flow faisait négocier les agents équipes sur des estimations agrégées en *story points*. Conséquence : leurs arguments étaient flous, leurs refus de portfolio non explicables, et la négociation entre équipes pouvait converger sur un consensus infaisable côté ressources et calendrier, un goulot interne (par exemple une ressource senior surchargée à 130 % sur le sprint 2) restait masqué derrière la moyenne SP.

Correction (DEC-004). Aligné sur l'approche en deux phases du Memetic NSGA-II [3] : on exécute d'abord une passe exploratoire globale qui produit des plannings concrets (assignation *story*→ressource, dates, charge par sprint), puis on donne ces plannings aux agents équipes pour qu'ils négocient sur données réelles. Un re-scheduling *warm-start* valide les contre-propositions avant convergence (DEC-005), conformément au schéma à deux temps de [12].

8.2 NSGA-II éclatant les features sur plusieurs équipes

Symptôme. Sans contrainte explicite, l'optimiseur produisait des plans techniquement excellents mais distribuant les *stories* d'une même *feature* sur 3-4 équipes différentes, violation flagrante du principe SAFe « *one team owns one feature* » [21]. Le processus réel admet ce découpage (un epic est parfois réparti en features sur plusieurs équipes, section 2.1) mais le traite comme exception négociée, jamais comme cas par défaut.

Correction (DEC-006). L'éclatement n'est pas interdit, il est fortement pénalisé : chaque *story* placée hors de l'équipe majoritaire de sa *feature* multiplie par 100 la contribution correspondante au score. L'optimiseur l'évite donc spontanément, tout en conservant la possibilité d'y recourir lorsque le découpage est légitime (*component teams* [21]). Une contrainte dure aurait rendu le problème infaisable sur ces cas limites.

8.3 Buffer 20 % SAFe non respecté à l'étape 3

Symptôme. La règle empirique SAFe (issue de la loi de Kingman [25] : >80 % utilisation ⇒ lead-time super-linéaire) est qu'on ne doit pas dépasser 80 % de charge prévue. Or l'étape 3 laissait passer le flow vers NSGA-II tant que la charge restait ≤ 110 %. NSGA-II partait alors sur un portfolio infaisable et trouvait « les moins pires » compromis au lieu d'arbitrer en amont.

Correction (DEC-016). Seuil bloquant abaissé à 80 % par défaut : tant qu'au moins une équipe dépasse ce seuil ou refuse son portfolio, le flow reste en HITL à l'étape 3. La règle est explicitée dans un bandeau citant Reinertsen [25].

Nuance terrain. L'entretien PO a confirmé qu'en pratique **chaque équipe définit son propre flow**, et que le pourcentage de capacité réellement dédiable au PI Planning varie selon la maturité, la topologie et la **charge externe** de l'équipe (volume de support, afflux de *Problem Change Requests* suite à une campagne de recette, etc.). Le 80 % du buffer SAFe est donc une *va-*

Réf.	Décision	Justification (sources académiques + observation terrain)	Statut
DEC-001	Spécialisation des équipes (enum + <i>fullstack</i> fallback)	[21] : un ART contient 5-15 équipes typées. <i>Entretien PO</i> : les ARTs réels sont organisés en équipes <i>back</i> / <i>front</i> / <i>data</i> / <i>devops</i> / <i>mobile</i> / <i>qa</i> avec une équipe transverse <i>fullstack</i> pour les sujets cross.	Assumé
DEC-002	<code>span_factor</code> pivot 2.0, tiré dans [1,8;2,2] par équipe	[25] loi de Kingman ($\approx 50\%$ utilisation $\Rightarrow$ lead-time $\approx 2\times$).	Assumé
DEC-004	NSGA-II exploratoire global puis raffinement	[3] (deux phases mémétique); [12] (HGA en deux temps).	Mesuré
DEC-005	Replanification à chaud des contre-propositions d'équipe	[12] : schéma à deux temps. Valide les transferts de <i>features</i> proposés par les équipes avant la convergence du <i>swarm</i> .	Assumé
DEC-006	Cohésion feature $\rightarrow$ équipe : pénalité $\times 100$	[21] : « one team owns one feature »; [15] : profils flexibles avec coût. <i>Entretien PO</i> : l'éclatement d'une feature sur plusieurs équipes est traité comme exception négociée, jamais comme cas par défaut.	Vérifié
DEC-007	Diversité Pareto par farthest-first Hamming	[6] : crowding multi-agent; [8] : opérateurs de congestion.	Assumé
DEC-008	Modèle <code>DecisionTrace</code> runtime	[7] : revue insistant sur la traçabilité combinatoire.	Vérifié
DEC-010	PI 12 semaines $\times$ 4 sprints de 3 semaines	[21] (PI 8-12 sem.); <i>entretien PO</i> : format 4×3 semaines effectivement utilisé (cohérent avec la <i>cadence</i> trimestrielle des Business Owners).	Assumé
DEC-014	Conseil IA + rééquilibrage HITL au step 3	[22] : intégration ML/MOEA pour aide à la décision; [21] : ROAM board. <i>Entretien PO</i> : les RTEs effectuent une <i>capacity review</i> explicite et négocient les retraits feature par feature.	Mesuré
DEC-015	Choix scénario RTE explicite à l'étape 5	[4] : max-regret pour décision multi-acteurs; [21] : revue Business Owner obligatoire. <i>Entretien PO</i> : la décision finale n'est jamais déléguée à un consensus mécanique, le RTE tranche en présence des Product Owners.	Vérifié
DEC-016	Buffer 80 % bloquant	[25] (Kingman); [21]. <i>Entretien PO</i> : la règle des 20 % de buffer (capacity load $\leq 80\%$) est appliquée strictement par les RTEs avant validation du PI.	Mesuré
DEC-018	Marge de sécurité par <i>story</i> , inspirée de la Critical Chain	[24]; repris dans [17]; [22] : ML+MOEA.	Assumé
C1	Tardivité pondérée WSJF (6 ^e obj.)	[14] : cas industriel multi-projets, <i>weighted tardiness</i> .	Assumé
C2	Seed WSJF/deadline urgency	[1] INSGA-II; [2] variante améliorée sur cas industriel.	Assumé
C3	Recherche locale mémétique	[3] : recherche locale <i>greedy</i> sur le front 0.	Mesuré

TABLE 2 – Décisions de conception majeures, avec leurs sources académiques et, pour les choix concernés, les éléments issus de l'entretien PO.

leur cible par défaut et non une constante universelle. Le seuil bloquant est paramétrable par équipe dans le modèle de données; il est exposé au RTE pour qu'il puisse abaisser ce seuil sur les équipes connues pour absorber beaucoup de travail hors PI.

8.4 Surcharge sans boucle de rééquilibrage

Symptôme. Quand l'étape 3 détectait une surcharge, le flow n'offrait **aucun mécanisme de rééquilibrage automatique** : la seule sortie possible était de poursuivre vers NSGA-II avec une allocation infaisable. L'algorithme partait donc sur un portfolio intrinsèquement surdimensionné et produisait des compromis sous-optimaux par construction.

Correction (DEC-014). Introduction d'un **conseil IA déterministe** (équation 14) qui ordonne les *features* candidates par impact croissant, et d'une **boucle de re-CNP** (Capacity Negotiation Protocol) bornée par la règle des 80 %. Suivant l'esprit du *survey* [22] sur l'intégration ML/MOEA en aide à la décision multi-objectifs, la boucle ne se ferme qu'une fois la

contrainte de capacité satisfaite. L'optimiseur reçoit alors un portfolio sain plutôt que de devoir compenser en aval.

8.5 Retraits de l'étape 3 ignorés par l'étape 4

Symptôme. La boucle de rééquilibrage ci-dessus produisait bien une liste de *features* retirées, mais l'étape 4 ne la recevait pas : l'optimiseur rechargeait l'intégralité du *backlog* de l'ART depuis la base. Le RTE voyait donc réapparaître dans le planning final des *features* qu'il venait explicitement de sortir du PI. Le symptôme était d'autant plus insidieux qu'aucune erreur n'était levée et que le planning produit avait toutes les apparences de la validité.

Correction. Les identifiants exclus sont transmis au chargement du contexte d'optimisation, qui restreint le périmètre en conséquence, et le planning ne peut plus contenir de *story* appartenant à une *feature* retirée. La propriété est verrouillée par un test automatisé du parcours complet : sans lui, la régression serait passée inaperçue, puisqu'elle ne se manifeste ni par une erreur

ni par un planning visiblement incohérent.

8.6 Auto-finalisation court-circuitant la décision finale

Symptôme. L'étape 5 auto-finalisait sur consensus dès que les agents équipes s'accordaient sur deux stratégies préférentielles ou moins. Le *flow* sautait alors directement de l'étape 3 à un état « PI Planning terminé » sans avoir matérialisé un choix explicite : le scénario « recommandé » par le système était considéré comme accepté par défaut.

Correction (DEC-015). Étape 5 rendue **toujours** HITL par construction du flow. Inspirée du *max-regret* de [4], l'orchestrateur calcule un consensus indicatif (majorité de votes, recommandé par le *score composite*, Pareto-optimal) mais ne déclenche jamais la transition vers l'étape 6 sans une décision explicite. Cela respecte la sémantique du HITL conditionnel : conditionnel à *un problème détectable algorithmiquement*, et non sur une décision business fondamentalement humaine [21].

Calage sur le déroulé réel d'un PI Planning. Le rituel observé chez l'interviewé se déroule sur deux journées avec une structure claire, qu'on cherche à reproduire dans l'outil :

- **Journée 1.** Chaque PO présente la proposition de son équipe ; les parties prenantes (RTE, CPO, EDM, équipes en dépendance) acceptent ou refusent. Un *partage de synthèse* en fin de J1 permet d'identifier les difficultés à anticiper pour J2.
- **Journée 2.** Les équipes affinent leurs engagements en intégrant les contre-propositions de la veille. Une *revue de validation finale* en fin de J2 acte le plan définitif.
- **Issue possible.** Il arrive que le plan *ne soit pas accepté* à la fin des deux jours : dans ce cas, d'autres réunions sont tenues au cours des premiers jours du PI lui-même pour faire converger les équipes et les parties prenantes vers un plan validé.

SAFESCALE modélise cette structure en deux phases côté front-end : les K scénarios sont d'abord présentés en mode *revue* (analogue à J1, chaque équipe vote, on identifie les points durs), puis le RTE effectue la *validation finale* (J2) à partir des votes et du conseil de l'évaluateur. L'absence de validation explicite en fin de J2 produit un état « PI Planning en attente d'arbitrage » plutôt qu'un état « terminé ».

8.7 Front Pareto donnant l'illusion du choix

Symptôme. Le front Pareto contenait quelques dizaines de solutions, dont beaucoup étaient des voisins d'un même optimum. Sélectionner $K = 5$ par score composite produisait des scénarios visuellement différents mais qualitativement identiques.

Correction (DEC-007). On force la **diversification** des scénarios proposés au RTE : au lieu de prendre les $K = 5$ meilleurs au score composite, on construit la liste un à un en prenant à chaque fois le scénario le plus différent de ceux déjà choisis. La distance utilisée est la *distance de Hamming* sur l'assignation feature→équipe (cf. section 6 pour le détail). À la suite de [6] (uniformité du front) et [8] (opérateurs de congestion).

8.8 Variance d'exécution et équipes en sous-activité

Constat terrain. La contrainte majeure observée aujourd'hui sur les ARTs reste la **surcapacité** : des équipes engagées au-delà de ce qu'elles peuvent absorber. Mais le retour PO décrit aussi le phénomène inverse : des équipes **en sous-activité** sur un PI, soit parce qu'elles ont une vitesse plus haute que prévu, soit parce que leur portefeuille s'est dégonflé entre la planification et l'exécution.

Symptôme côté optimiseur. Avec l'équation 2 seule, deux problèmes apparaissent :

- une équipe planifiée à 80 % voit son planning se compacter sur 2 sprints alors qu'elle dispose de 4, sans aucune marge pour la variance d'exécution ;
- l'optimiseur ne propose rien pour valoriser la capacité libérée par une équipe en sous-activité.

Correction (DEC-018), pour la variance d'exécution. Insertion de **marges de sécurité** ciblées sur les *stories* à variance élevée (équation 7), selon le principe de Goldratt [24] et le *survey* ML+MOEA [22]. Variante du *fuzzy RCPSP* [17] qui convertit l'incertitude en objectifs ; ici on la convertit en buffers locaux.

Piste pour la sous-activité (à explorer). Quand une équipe est sous-utilisée, deux leviers métier sont envisageables, en complément des buffers :

- **Anticiper des stories du PI suivant** : remonter dans le backlog des features prévues pour le PI N+1 et les ajouter de manière opportuniste si la compétence colle ;
- **Prendre en charge des stories d'autres équipes** surchargées, à condition que la spécialisation le permette (*cross-team support*, typiquement par une équipe *fullstack*).

Ces deux mouvements sont déjà partiellement modélisables dans SAFESCALE : la première s'inscrit dans le mécanisme de retrait/ajout de features de l'étape 3, la seconde dépend du modèle de spécialisation déjà décrit en section 4. Leur intégration explicite est listée comme perspective P7 (section 12).

8.9 Goulot capacité par compétence non instrumenté

Symptôme. Un goulot sur une compétence rare (*ml_ops*, *mobile*, *devops*) ne ressort jamais comme signal isolé : la surcharge se manifeste à l'échelle équipe et noie l'origine. Le RTE retire des *features* « au hasard » sans savoir laquelle débloque le goulot.

Correction. Service `skill_bottleneck` qui calcule *demand*, *supply* et *gap* par compétence, identifie les équipes couvrantes, et pondère les *features* candidates au retrait par leur effort libéré sur la compétence en goulot. Cas extrême : compétence non couverte par aucune équipe $\Rightarrow$ bonus +5 au score de retrait des *features* qui la requièrent. Cette modélisation s'appuie sur la notion de *flexible resource profile* de [15].

9 Tests et résultats

Les sections précédentes décrivent un système et justifient ses choix. Celle-ci cherche à savoir ce qu'il vaut réellement. La différence compte : jusqu'ici, tout ce qui a été affirmé reposait sur la littérature, sur l'entretien

avec le Product Owner, ou sur du bon sens. Rien n'avait été mesuré.

9.1 Les quatre questions

Un outil d'aide à la décision peut rater sa cible de plusieurs façons, et elles sont indépendantes les unes des autres. Il peut proposer des plannings impossibles à exécuter. Il peut en proposer d'exécutables, mais pas meilleurs que ce qu'on obtiendrait en rangeant les tâches à la main. Il peut enfin reposer sur des raffinements d'algorithme qui, une fois mesurés, ne changent rien. D'où quatre questions, posées dans l'ordre où une réponse négative rendrait les suivantes sans objet :

- **Q1. Les plannings proposés sont-ils réalisables ?** Une *story* ne peut pas commencer avant celle dont elle dépend, une personne ne peut pas travailler sur deux *stories* le même jour, et rien ne doit avoir été oublié en route.
- **Q2. Les deux améliorations apportées à NSGA-II servent-elles vraiment ?** Il s'agit des plannings de départ construits par des règles métier (C2) et de la retouche des meilleurs plannings en fin de course (C3).
- **Q3. Le système fait-il mieux qu'une méthode simple ?** On le compare à un rangement des *stories* par ordre de priorité, puis on regarde ce qu'il reste à gagner avant d'atteindre la limite physique du problème.
- **Q4. La règle des 80 % change-t-elle le résultat ?** Autrement dit, le blocage de l'étape 3 sert-il à quelque chose, ou n'est-ce qu'une formalité ?

9.2 Comment la campagne est menée

Les jeux de données. La campagne porte sur 5 jeux de données, produits par le générateur avec des graines différentes. Un seul ne suffirait pas : on décrirait alors les particularités d'un tirage plutôt que le comportement de l'algorithme. Le jeu de graine 42 sert de référence quand on entre dans le détail.

Jeu de données	Stories	SP	Cap.	Ratio	Préc.	Vivier	BI
42	112	801	625	1,28	207	4,0	281
43	131	929	710	1,31	246	3,7	183
44	131	952	640	1,49	223	3,7	204
45	129	892	645	1,38	436	3,9	169
46	129	971	610	1,59	281	3,8	224

Ratio : ce qui est demandé aux équipes divisé par ce qu'elles peuvent absorber. Au-dessus de 1, le PI est trop chargé. **Préc.** : nombre de contraintes « celle-ci avant celle-là » entre *stories*; quand deux *features* se suivent, toutes leurs *stories* héritent de la contrainte, d'où un nombre élevé. **Vivier** : combien de personnes peuvent prendre une *story*, en moyenne; plus c'est bas, moins on a le choix. **BI** : durée en jours sous laquelle aucun planning ne peut descendre.

Deux colonnes méritent un mot. Le **ratio** dépasse 1 partout : le générateur crée volontairement un PI trop chargé, puisque c'est justement la situation que le PI Planning doit trancher. Et le nombre de **précédences** est bien plus élevé qu'on ne l'imaginerait en comptant les dépendances saisies : quand une *feature* doit attendre une autre, ce sont toutes les *stories* de la première qui doivent attendre toutes celles de la seconde. Ne compter que les dépendances saisies donnerait l'impression d'un problème presque sans contraintes d'ordre.

Les trois versions comparées. On compare trois versions du même algorithme, chacune ajoutant une brique à la précédente :

- *standard* : NSGA-II tel qu'il est décrit dans la littérature, qui démarre sur des plannings tirés au hasard ;
- *+seeds* : le même, mais qui démarre sur les six plannings construits par des règles métier (section 6) ;
- *+mémétique* : le précédent, plus la retouche des meilleurs plannings en fin de course.

Les trois disposent du même budget de calcul : 60 plannings menés en parallèle, 120 générations. Si on leur donnait des budgets différents, on mesurerait le temps de calcul et non l'apport de la brique ajoutée.

Pourquoi répéter. L'algorithme comporte une part de hasard : deux exécutions identiques ne donnent pas le même résultat. Chaque version est donc lancée 10 fois sur chaque jeu de données, soit 50 mesures par version.

Ce que l'on mesure. Le système ne propose pas un planning mais un lot de plannings, chacun représentant un compromis différent entre les six objectifs. Ce que la section 6 appelle le *front de Pareto* est désigné ici plus simplement par « le lot de plannings » : il s'agit du même objet.

Une image aide à comprendre ce qui est mesuré. Chaque planning peut être vu comme un point dans un espace à six dimensions, une par critère : sa durée sur un axe, son risque sur un autre, sa marge sur un troisième, et ainsi de suite. Un lot de plannings forme alors un nuage de points dans cet espace, et juger le lot revient à juger la forme de ce nuage. Occupe-t-il une large zone, ou reste-t-il groupé dans un coin ? Se tient-il près de la région où les six critères sont au mieux ? Ses points sont-ils répartis, ou tassés les uns sur les autres ? Les quatre indicateurs qui suivent répondent chacun à l'une de ces questions ; leurs noms techniques sont donnés entre parenthèses, pour le lecteur qui voudrait les retrouver dans la littérature.

- la **couverture**, appelée *hypervolume* dans la littérature : quelle part de l'éventail des compromis possibles le lot occupe-t-il réellement ? Plus il en couvre, plus le RTE a de choix devant lui.
- la **distance au meilleur** (IGD+ [28]) : à quelle distance le lot se trouve-t-il du meilleur qu'on ait obtenu, toutes versions et toutes exécutions confondues ? Ce « meilleur » n'est le résultat d'aucune exécution en particulier : on le fabrique après coup, en réunissant ce que chacune a produit de mieux et en ne conservant que ce que rien ne surpasse. Aucune version ne l'atteint donc entièrement ; il sert d'étalon commun.
- la **régularité** (*spread*) : les plannings sont-ils espacés régulièrement, ou groupés en quelques paquets séparés par des trous ? Un RTE qui n'a le choix qu'entre trois variantes presque identiques n'a pas vraiment le choix.
- le **taux de recouvrement** (C-metric [27]) : en comparant deux lots planning par planning, l'un fait-il systématiquement au moins aussi bien que l'autre ?

Un point de méthode conditionne tout ce qui suit. Mesurer une couverture suppose de fixer d'abord le cadre

dans lequel on mesure : un plancher, donné par la meilleure valeur observée sur chaque critère, et un plafond, donné par la pire. La couverture est alors la part de ce cadre que le nuage occupe.

Si l'on redéfinit ce cadre pour chaque lot, on compare des pourcentages calculés sur des cadres différents, et le classement peut s'inverser : un lot très étalé fixe lui-même un cadre très large, et n'en occupe donc qu'une fraction ; un lot resserré remplit au contraire presque entièrement le cadre étroit qu'il s'est donné. Le second obtiendrait le meilleur score, soit exactement l'inverse de ce que l'on cherche. La première version du code commettait cette erreur. Le cadre est donc désormais fixé une fois par jeu de données, sur l'ensemble de ce que les trois versions ont produit, et tous les lots y sont mesurés.

Décider si un écart compte. Comparer deux moyennes ne suffit pas. L'algorithme comportant une part de hasard, une moyenne plus haute peut n'être qu'une suite de tirages favorables. Chaque comparaison s'accompagne donc de deux chiffres, qui répondent à deux questions distinctes.

Le premier est le **risque que ce soit le hasard**. On se demande : si les deux versions se valaient réellement, quelle serait la probabilité de voir apparaître un écart au moins aussi grand que celui qu'on observe ? Quand cette probabilité tombe sous 5 %, on écarte l'explication par le hasard.

Le second est l'**ampleur de l'écart**. On confronte chaque exécution d'une version à chaque exécution de l'autre, et on compte la proportion de duels que la première remporte. À 0,5, les deux versions se valent : une fois sur deux, chacune l'emporte. À 0,8, la première gagne quatre duels sur cinq.

Les deux chiffres sont nécessaires, et pour des raisons opposées. Le premier, seul, ne dit rien de l'importance : en multipliant les exécutions, on finit toujours par rendre « improbable » un écart minuscule. Le second, seul, ne dit rien de la fiabilité : un écart peut sembler large et n'être qu'un accident de tirage.

Pour le lecteur qui voudrait vérifier : le premier chiffre est la p -value d'un test de Mann-Whitney, le second la statistique A_{12} de Vargha-Delaney [29, 30]. Les probabilités sont ajustées par Holm-Bonferroni, parce que les trois comparaisons portent sur les mêmes données et que multiplier les tests augmente mécaniquement le risque d'en voir un « passer » par chance.

Rejouer la campagne. Tout se relance avec une seule commande (`manage.py run_experiments`). Elle travaille dans une base de données temporaire, créée puis supprimée, et écrit à la fois les résultats bruts et les tableaux de cette section. Aucun chiffre n'est donc recopié à la main : relancer la campagne les régénère, et le moindre écart se voit immédiatement. Comptez environ 62 minutes.

9.3 Q1. Les plannings proposés sont-ils réalisables ?

Un lot de plannings impossibles à exécuter ne vaut rien, et l'algorithme ne le signalerait pas : il optimise

ce qu'on lui demande de mesurer, pas ce qu'on a oublié de vérifier. Chaque planning proposé est donc repris et contrôlé sur quatre points : une *story* ne démarre jamais avant la fin de celles dont elle dépend, marge de sécurité comprise (celle insérée après les stories les plus risquées, section 6.4) ; personne ne travaille sur deux *stories* en même temps ; chaque *story* est bien exécutée par quelqu'un de l'équipe qui en a la charge ; et aucune n'a été oubliée.

Sur l'ensemble de la campagne, 100,0 % des plannings respectent ces quatre points. Ce n'est pas un coup de chance : c'est une conséquence directe de la façon dont un planning est construit. Le programme place chaque *story* au plus tôt, une fois ses prédécesseurs terminés et sa ressource libre ; et les opérations qui font évoluer les plannings d'une génération à l'autre refusent tout réordonnement qui casserait l'ordre des dépendances. Ce point est vérifié séparément au démarrage et après chaque type de modification : un taux global de 100 % pourrait sinon cacher une opération qui n'est jamais déclenchée.

9.4 Q2. Les deux améliorations servent-elles vraiment ?

La question se pose simplement. NSGA-II est un algorithme publié et éprouvé ; nous lui avons ajouté deux choses. D'abord des **plannings de départ** construits par des règles métier (C2), au lieu de démarrer sur des plannings tirés au hasard. Ensuite une **retouche finale** (C3), qui reprend les meilleurs plannings du lot et fouille autour d'eux. Chacune représente du travail : du code à écrire, à tester, à maintenir. Servent-elles à quelque chose ?

Pour y répondre, on compare les trois versions sur les indicateurs définis plus haut. Le premier tableau donne les valeurs moyennes.

Version	Couverture	Dist. au meilleur	Régularité	Non départagés	Réalisables	Temps (s)
NSGA-II standard	0,417	0,090	0,777	100,0 %	100,0 %	6,1
+ seeds (C2)	0,409	0,108	0,761	100,0 %	100,0 %	6,1
+ mémétique (C3)	0,455	0,081	0,774	100,0 %	100,0 %	7,1

5 jeux de données $\times$ 10 exécutions chacun, soit 50 mesures par version. **Couverture** : part de l'éventail des compromis possibles que le lot occupe réellement ; plus c'est haut, plus le RTE a de choix. **Dist. au meilleur** : à quelle distance le lot se trouve du meilleur qu'on ait vu, toutes versions confondues ; plus c'est bas, mieux c'est. **Régularité** : les plannings sont-ils espacés régulièrement, ou groupés en paquets avec des trous ; plus c'est bas, mieux c'est. **Non départagés** : part des 60 plannings du lot qu'aucun autre ne surpasse ; à 100 %, le critère de tri de NSGA-II ne trie plus rien. **Réalisables** : part des plannings réellement exécutables.

Une ligne se détache et une autre déçoit. La version *+mémétique* obtient la meilleure couverture (0,455) et la plus faible distance au meilleur (0,081). La version *+seeds*, elle, ne fait pas mieux que le standard : sa couverture est même légèrement inférieure (0,409 contre 0,417).

Reste à savoir si ces écarts sont réels ou s'ils tiennent au hasard des tirages. C'est l'objet du second tableau.

Comparaison	A	B	Risque hasard	Ampleur	Effet
+ seeds (C2) vs NSGA-II standard	0,409	0,417	0,9067	0,507	non significatif
+ mémétique (C3) vs NSGA-II standard	0,455	0,417	0,0065	0,678	moyen
+ mémétique (C3) vs + seeds (C2)	0,455	0,409	0,0091	0,665	moyen

A et **B** : couverture moyenne des deux versions comparées. **Risque hasard** : probabilité d'observer un tel écart alors que les deux versions se valent ; en dessous de 0,05, on écarte l'explication par le hasard. **Ampleur** : 0,5 signifie que les deux versions se valent, et plus on s'en éloigne, plus l'écart pèse. Les deux

sont nécessaires : un écart minuscule finit toujours par devenir « improbable » si l'on multiplie les exécutions. (Test de Mann-Whitney, probabilités ajustées par Holm-Bonferroni pour tenir compte des trois comparaisons.)

Le verdict est net dans les deux sens.

Les plannings de départ (C2) n'améliorent pas le résultat final. L'écart avec la version standard est non significatif : le risque que ce soit le hasard vaut 0,9067, très au-dessus du seuil de 5 %, et l'ampleur 0,507, c'est-à-dire à peu près un duel gagné sur deux. Autrement dit, partir de six plannings construits par des règles métier ne donne pas un meilleur lot à l'arrivée.

L'explication tient à la variété. Ces six plannings sont tous *bons* et tous *semblables*. Ils ne pèsent que six plannings sur 60, mais comme ils sont meilleurs que les autres, ils sont retenus en priorité dès les premières générations, et leurs affectations se retrouvent dans les plannings issus des croisements, génération après génération, jusqu'à tirer toute la population vers la même zone. C'est le dilemme habituel de ce type d'algorithme : ce qu'on gagne à bien démarrer, on le perd à moins explorer. Sur 120 générations, les deux effets s'annulent.

Jeu de données	NSGA-II standard	+ seeds (C2)	+ mémétique (C3)
42	0,534	0,485	0,532
43	0,432	0,439	0,489
44	0,386	0,386	0,435
45	0,362	0,425	0,431
46	0,372	0,307	0,390

Couverture moyenne sur 10 exécutions, jeu de données par jeu de données ; le meilleur de chaque ligne est en gras. Ce détail montre ce que la moyenne cache : une configuration qui gagne partout n'a pas la même valeur qu'une configuration qui gagne ici et perd là.

Le détail jeu par jeu montre que « pas d'écart en moyenne » ne veut pas dire « pas d'effet ». Les plannings de départ font nettement mieux que le standard sur le jeu 45 (0,425 contre 0,362) et nettement moins bien sur le jeu 46 (0,307 contre 0,372). L'effet existe donc, mais il s'inverse selon la forme du *backlog*, et la moyenne ne l'annule que parce que les bons et les mauvais cas se compensent. Si l'on avait conclu sur un seul jeu de données, on aurait pu présenter C2 comme un gain ou comme une perte, selon celui qu'on aurait choisi.

La retouche finale (C3), elle, apporte un vrai gain. Sa couverture atteint 0,455 contre 0,417 pour le standard, avec un risque de hasard de 0,0065 et une ampleur de 0,678, soit environ deux duels gagnés sur trois. L'écart avec *+seeds* est du même ordre (0,0091 et 0,665). Surtout, l'avantage est **stable** : cette version gagne sur quatre jeux de données sur cinq, et n'est devancée que d'un millièm sur le cinquième. Le coût est modeste : 7,1 s par exécution contre 6,1 s.

La figure 3 montre d'où vient ce gain. Elle suit la couverture du lot au fil des générations, moyennée sur les cinq jeux de données. Trois choses s'y lisent.

D'abord, les deux versions qui démarrent sur des plannings construits partent réellement devant : dès la première génération, elles couvrent 0,225 contre 0,196

pour le standard. Le travail investi dans les règles métier n'est donc pas sans effet, il est simplement précoce.

Ensuite, la courbe des plannings de départ plafonne. Le standard la rattrape vers la cinquantième génération et la dépasse ensuite : c'est la perte de variété décrite plus haut, qui se voit ici directement.

Enfin, la version mémétique garde son avance du début à la fin et la creuse même. À chaque passage, la retouche reprend les plannings les plus isolés du lot et va chercher des compromis que les deux autres versions ne trouvent jamais.

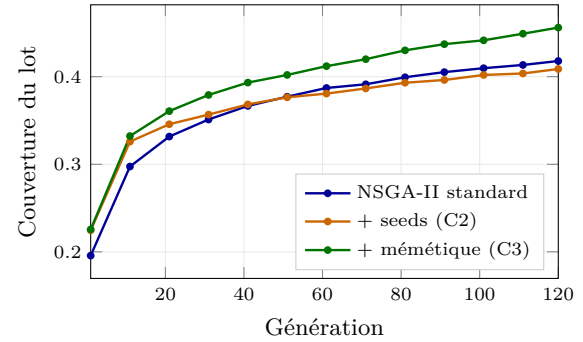


FIGURE 3 – Couverture du lot de plannings au fil des générations, moyennée sur les 5 jeux de données et les 10 exécutions de chacun. C'est le même indicateur que la première colonne du tableau des résultats, suivi cette fois pas à pas : les valeurs de la dernière génération sont celles du tableau. Les plannings de départ partent devant puis se font dépasser ; la retouche finale, elle, garde son avance.

A\B	NSGA-II standard	+ seeds (C2)	+ mémétique (C3)
NSGA-II standard	–	0,360	0,243
+ seeds (C2)	0,107	–	0,115
+ mémétique (C3)	0,194	0,358	–

Mesuré sur le jeu de données de référence. $C(A, B)$ = part des plannings de B pour lesquels A propose au moins aussi bien. On lit les deux sens ensemble : si A couvre beaucoup de B et B peu de A , alors A est meilleure ; si les deux valeurs sont proches, les deux explorent simplement des compromis différents.

Un dernier tableau compare les lots deux à deux, sur le jeu de référence, et confirme la lecture précédente par un autre chemin. Entre *standard* et *+mémétique*, aucune des deux ne prend l'ascendant : la première fait au moins aussi bien sur 24,3 % des plannings de la seconde, qui le lui rend sur 19,4 %. Elles proposent simplement des compromis différents, et l'avantage de la version mémétique vient de l'étendue de ce qu'elle propose, pas d'une supériorité planning par planning. Sur ce jeu-là, en revanche, *+seeds* est devancée dans les deux sens : les deux autres versions font au moins aussi bien sur 36,0 % et 35,8 % de ses plannings, quand elle n'y parvient que sur 10,7 % et 11,5 % des leurs.

Avec six objectifs, l'algorithme n'arrive plus à départager. Dans les trois versions, 100 % de la population finale est jugée « non surpassée » : aucun planning n'est meilleur qu'un autre sur les six critères à la fois. Ce n'est pas une bonne nouvelle, mais c'est un phénomène connu quand on empile les objectifs [9, 11] : plus il y en a, plus il devient rare qu'un planning batte

un autre partout, et le critère de tri finit par ne plus rien trier. L'algorithme se rabat alors entièrement sur la *crowding distance*, un mécanisme prévu pour garder de la variété, pas pour classer.

Cela éclaire d'ailleurs le résultat précédent : si le tri ne fait plus son travail, bien démarrer sert moins que bien explorer, ce que la retouche finale apporte et ce que les plannings de départ n'apportent pas. C'est l'argument chiffré le plus net en faveur du passage à NSGA-III (section 12, P2), qui remplace justement ce critère de tri par un autre, conçu pour rester utile au-delà de trois ou quatre objectifs.

9.5 Q2 bis. Et si l'on donnait moins de temps de calcul ?

Le résultat précédent est gênant : les plannings de départ demandent un travail réel (six heuristiques métier écrites, testées, maintenues) et la mesure ne leur trouve aucun effet. Avant de les déclarer inutiles, il reste une hypothèse à écarter, et la figure 3 la désigne : l'avance des plannings de départ est réelle, mais elle ne dure que le temps des premières générations. Si tout tient à cela, cette avance devrait rester visible quand on arrête l'algorithme tôt, avant que le standard n'ait eu le temps de la combler.

La question n'est pas théorique. Pendant la discussion, l'application ne laisse pas tourner l'optimisation pendant 120 générations : le RTE attend une réponse, et l'étape 4 s'arrête à 60. C'est donc à ce budget-là qu'il faut juger C2, pas au budget confortable de la campagne principale. On relance donc les trois versions à quatre budgets croissants.

Génération	NSGA-II standard	+ seeds (C2)	+ mémétique (C3)	Écart seeds / standard
20	0,336	0,352	0,365	$p = 0,1234$, $A_{12} = 0,59$ (non significatif)
40	0,372	0,375	0,400	$p = 0,5304$, $A_{12} = 0,54$ (non significatif)
60	0,394	0,389	0,420	$p = 0,8093$, $A_{12} = 0,51$ (non significatif)
120	0,426	0,419	0,466	$p = 0,8254$, $A_{12} = 0,51$ (non significatif)

Couverture moyenne sur 10 exécutions et 5 jeux de données, à budget de calcul croissant. La dernière colonne compare les plannings de départ (C2) à la version standard au même budget. L'application s'arrête à 60 générations pendant la discussion : c'est cette ligne-là qui décide si C2 sert en conditions réelles.

Le balayage ne sauve pas C2. À 20 générations, l'écart va bien dans le sens attendu (0,352 contre 0,336, $A_{12} = 0,59$), mais il reste non significatif ($p = 0,1234$) : la tendance existe, elle n'est pas établie. Et à 60 générations, le budget que l'application utilise réellement, il n'en reste rien du tout ($A_{12} = 0,51$, soit l'équivalent d'un tirage à pile ou face).

L'hypothèse était donc à moitié juste. Il subsiste bien une trace d'avantage au budget le plus serré, mais elle est trop faible pour qu'on puisse la distinguer du hasard, et elle a disparu avant même d'atteindre les 60 générations de l'application. Nous n'avons trouvé aucun régime dans lequel C2 se justifie par la qualité du lot produit. Le choix est donc reclassé *assumé* dans le tableau 2, et repris en section 9.9 : il n'est pas condamné, il n'est simplement pas démontré.

Le tableau apporte en revanche une confirmation utile sur C3 : la retouche finale arrive en tête aux quatre budgets, y compris le plus serré. Son intérêt ne dépend donc pas du temps qu'on lui laisse, ce qui n'était pas acquis d'avance pour une technique de raffinement : on aurait pu craindre qu'elle n'ait pas le temps d'agir.

9.6 Q3. Fait-on mieux qu'une méthode simple ?

Comparer les trois versions entre elles ne dit pas si l'ensemble vaut mieux qu'une méthode simple. On confronte donc le système à quatre façons élémentaires de planifier : ranger les *stories* par valeur business, par échéance, par longueur de chaîne de dépendances, ou en équilibrant la charge, puis les placer dans cet ordre. Ces quatre méthodes sont évaluées sur le même jeu de données, **avec exactement le même calcul de planning et les mêmes formules d'objectifs** : l'écart observé ne peut donc pas venir d'une différence de mesure, seulement de la façon de chercher.

Reste à savoir ce qu'un bon résultat veut dire. La meilleure durée possible est inconnue : c'est justement ce qui rend le problème difficile. On calcule donc trois durées en dessous desquelles aucun planning ne peut passer, et on garde la plus contraignante : la plus longue chaîne de *stories* qui doivent se suivre, la charge d'une équipe répartie au mieux entre ses membres, et surtout le cas où une seule personne sait faire tout un lot de *stories*. Elle les enchaîne alors une par une, et cette durée-là est incompressible.

Méthode	Durée (j)	Dépass. capacité (SP)	Retard pondéré
Glouton WSJF	282	307	89,2
Glouton deadline	282	307	89,6
Glouton chemin critique	282	295	74,5
Glouton équilibrage	282	307	89,2
NSGA-II standard	281	50	70,8
+ seeds (C2)	281	54	67,4
+ mémétique (C3)	281	55	66,6
Borne inférieure	281	—	—

Jeu de données de référence (graine 42). Une méthode simple ne donne qu'un planning, NSGA-II en donne plusieurs : on retient pour lui le meilleur de son lot sur chaque colonne. La borne inférieure est la plus contraignante des trois durées incompressibles : la chaîne de dépendances la plus longue (101 j), la personne seule à savoir faire un lot de *stories* (281 j), et la charge d'une équipe répartie au mieux (114 j).

Le tableau se lit en deux temps.

Sur la durée, il n'y a rien à gagner. Le système termine en 281 jours, les méthodes simples en 282, et la limite calculée plus haut vaut 281 jours : le système est donc exactement sur cette limite, et les méthodes simples à un jour d'elle. Ce n'est pas un échec de l'algorithme, c'est une caractéristique du jeu de données : 19 *stories* ne peuvent être prises par une seule personne, et celle qui en concentre le plus doit les enchaîner l'une après l'autre, soit 281 jours à elle seule, bien plus que la plus longue chaîne de dépendances (101 jours) ou que la charge d'une équipe (114 jours). Aucun planning ne peut faire mieux. Et c'est un résultat utile en soi : il dit que **pour raccourcir ce PI, il faut former ou recruter sur les compétences rares, pas améliorer l'algorithme.**

Cette égalité parfaite est propre au jeu de référence, et il serait malhonnête d'en faire une règle générale. Sur les 5 jeux, l'écart à la limite va de 0 % à 63 %, pour une moyenne de 24 %. Attention toutefois à ne pas lire cet écart comme la marge de progression du système : cette limite est une garantie de type « on ne peut pas faire moins », pas la meilleure durée réellement atteignable. Un écart important peut donc tout aussi bien

signifier que la limite est trop basse. Les deux jeux où elle colle au résultat (0 et 1 % d'écart) sont justement ceux où une personne seule, ou une équipe entière, sature ; ailleurs, aucune des trois limites ne décrit ce qui bloque vraiment.

Sur le reste, l'écart est considérable. Le dépassement de capacité des équipes tombe de 295 SP pour la meilleure méthode simple à 50 SP, soit 5,9 fois moins ; et le retard sur les *features* à forte valeur business recule de 10,5 %. C'est là que se situe l'apport réel du système sur ce jeu de données : non pas finir plus tôt, ce qui est impossible, mais répartir la charge sans faire exploser les équipes et livrer d'abord ce qui compte le plus.

9.7 Q4. La règle des 80 % change-t-elle le résultat ?

La section 8 défendait le blocage de l'étape 3 par un argument théorique : au-delà de 80 % de charge, les délais s'allongent beaucoup plus vite que la charge elle-même [25]. L'expérience mesure ce que cette règle change en pratique.

Un petit programme rejoue l'arbitrage qu'on attend du RTE : tant qu'une équipe dépasse 80 % de charge, il retire la *feature* la moins utile au sens de l'équation 14 : faible valeur business et peu de *features* bloquées derrière elle. On replanifie ensuite ce qui reste, et on compare.

Jeu de données	Features ret.	Charge max	Durée (j)	Dépass. capacité (SP)
42	10	1,60 → 0,79	281 → 128	55 → 3
43	11	1,75 → 0,73	255 → 115	27 → 0
44	13	1,91 → 0,77	206 → 85	76 → 2
45	12	2,32 → 0,77	275 → 147	58 → 2
46	12	1,86 → 0,75	267 → 88	91 → 2

À gauche de la flèche, le PI tel qu'il arrive à l'étape 3 ; à droite, le même PI après avoir retiré les *features* les moins utiles, une par une, jusqu'à ce que plus aucune équipe ne dépasse 80 % de charge.

L'effet est massif et va dans le même sens sur les cinq jeux de données. Sur celui de référence, 10 *features* sortent du PI, la charge de l'équipe la plus chargée tombe de 1,60 à 0,79, la durée de 281 à 128 jours et le dépassement de capacité de 55 à 3 SP.

Deux conclusions, dont une inconfortable. D'abord, la règle sert bel et bien à quelque chose : sans elle, l'étape 4 rend un planning qui a l'air complet, puisque toutes les *features* y sont, mais qui demanderait plus de quatre fois la durée du PI. Le blocage est ce qui empêche le système de faire prendre à l'ART un engagement intenable. Ensuite, l'ampleur du retrait (521 SP sur 801, près des deux tiers du *backlog*) traduit surtout un jeu de données volontairement surchargé ; ce n'est pas une prédiction de ce qu'un RTE ferait. Dans un vrai PI Planning, on découpe aussi des *features*, on en décale au PI suivant, on renégocie le périmètre : trois leviers que le programme ne connaît pas. Ce que l'expérience établit, c'est donc le **sens** de l'effet et son ordre de grandeur, pas le nombre exact de *features* à retirer.

9.8 Ce que ces résultats ne prouvent pas

Les résultats précédents tiennent dans un périmètre précis. Mieux vaut le dire que le laisser deviner.

Les données sont fabriquées. Elles sont dimensionnées d'après un entretien avec un Product Owner (section 4), mais un générateur reproduit ce qu'on lui a décrit, pas les irrégularités du terrain. Rien ne garantit que les conclusions tiennent telles quelles sur un ART réel, en particulier pour les dépendances, plus régulières ici que sur un vrai *Program Board*.

Le risque est calculé par une règle, pas appris. Le mode apprentissage automatique existe, mais il n'a pas pu être entraîné faute de données historiques réelles. L'objectif « risque » mesure donc une règle que nous avons écrite, pas une régularité constatée sur le terrain.

Les durées absolues sont approximatives. Elles reposent sur le facteur d'étalement (équation 2), qui est lui-même une estimation. S'il est un peu faux, toutes les durées se décalent dans la même proportion : les comparaisons entre versions restent valables, mais les nombres de jours sont à prendre comme des ordres de grandeur.

Certains écarts restent indécidables. Les tests statistiques reposent sur 50 mesures par version. C'est assez pour détecter un écart franc, pas pour trancher un écart faible. Quand un écart est déclaré « non significatif », cela veut dire « non démontré ici », et surtout pas « démontré équivalent ».

Ce qui n'a pas pu être testé. La comparaison **système contre PO humain**, annoncée comme second protocole, n'a pas eu lieu : elle demande qu'un PO se rende disponible pour une session complète de PI Planning, ce qui n'a pas été possible. C'est la principale lacune de cette évaluation, car elle seule dirait si le système est réellement utile à ceux qui s'en servent, et pas seulement correct. L'autre protocole annoncé, le nom donné automatiquement à chaque scénario, a en revanche été mis en place et est vérifié par les tests du parcours complet (section 10).

9.9 Ce qui reste assumé sans preuve

La colonne **Statut** du tableau 2 distingue trois cas : *mesuré*, *vérifié* et *assumé*. Cette dernière catégorie mérite d'être reprise ici, car c'est la plus fragile et la plus facile à passer sous silence.

Un choix *assumé* n'est pas un choix arbitraire : il s'appuie sur la littérature ou sur l'entretien avec le Product Owner. Mais rien dans ce mémoire ne montre qu'il produit un meilleur résultat qu'une autre option. Cinq méritent d'être signalés.

- **Le facteur d'étalement** (DEC-002). Sa valeur pivot de 2,0 vient de la loi de Kingman à 50 % d'utilisation. Aucune donnée réelle ne l'a confirmée ici, et il conditionne toutes les durées annoncées.
- **La sélection des cinq scénarios présentés au RTE** (DEC-007). On choisit les plus éloignés les uns des autres pour éviter d'en proposer cinq quasi identiques. C'est raisonnable, mais on n'a pas comparé cette règle à d'autres façons de choisir, ni demandé à un RTE laquelle l'aide le plus.

- **La marge de sécurité par *story*** (DEC-018). Le principe vient de la *Critical Chain* de Goldratt, mais la formule qui calcule sa taille, comme les poids des cinq facteurs qu'elle combine, ont été fixés par nos soins.
- **Les poids du score composite** (DEC-015). Ils désignent le scénario recommandé par défaut. Nous les avons fixés à la main, sans les comparer à d'autres jeux de poids ni demander à un RTE lequel correspond à son arbitrage.
- **La tardivité pondérée par WSJF** (C1). Ce sixième objectif est repris d'un cas industriel [14]. On n'a pas mesuré ce que le système donnerait sans lui, et la section 9.4 montre justement qu'ajouter des objectifs a un coût : au-delà de quatre ou cinq, le critère de tri de NSGA-II cesse de fonctionner.

À ces cinq cas s'en ajoute un sixième, et c'est le plus instructif, parce qu'on a essayé de l'en faire sortir. **Les plannings de départ** (C2) ont fait l'objet d'une expérience dédiée (section 9.5), montée précisément pour vérifier s'ils servaient à budget serré. La réponse est non, ou plus exactement : non démontré, à aucun des quatre budgets testés.

Nous les conservons malgré tout, et il vaut mieux dire pourquoi que de laisser croire à un oubli. Trois raisons. Ils ne coûtent rien : le temps de calcul est identique à celui de la version standard. Leur avance de départ est bien réelle, la figure 3 la montre ; elle ne survit simplement pas jusqu'au bout. Et le détail jeu par jeu montre qu'ils font parfois nettement mieux, sur un des cinq jeux de données, sans qu'on sache prévoir sur quelle forme de *backlog* cela se produit. Un gain occasionnel et gratuit reste préférable à pas de gain du tout. Mais c'est un pari sur l'usage, pas un résultat mesuré, et il doit être lu comme tel.

Énoncer cette liste n'est pas un aveu de faiblesse mais une précaution de méthode : elle indique précisément où porter l'effort si le travail devait être poursuivi, et elle empêche de lire l'ensemble de ce mémoire comme si tout y avait été démontré.

10 Retours sur la solution

10.1 Observations qualitatives

Les retours qui suivent sont **qualitatifs** : ils proviennent de sessions de démonstration menées en interne sur le dataset de référence. Ils complètent l'évaluation chiffrée de la section 9 sans s'y substituer : ce qui est décrit ici relève de l'usage du système, pas de la qualité des plannings qu'il produit.

Lisibilité du flow. Découper le processus en cinq étapes nommées (*Préparation, Dépendances, Capacité & allocation, Planification, Choix du scénario*) rend chaque arbitrage plus facile à suivre. On peut reprendre une session étape par étape et expliquer ce qui s'est passé, ce qui est utile pour la défense.

Effet du buffer 80 %. Tant que le seuil bloquant n'existait pas, l'étape 3 se traversait vite et sans vraie discussion. En la rendant bloquante, elle devient un moment de négociation : le RTE voit explicitement pourquoi il doit retirer des *features*, puisque la règle est affichée.

Conseil de retrait. Le panneau de retrait au step 3 ordonne les *features* candidates par score d'impact (équation 14). En pratique, il met en avant des arbitrages peu intuitifs, par exemple qu'une *feature* à WSJF moyen sans *deadline* libère plus de marge qu'une *feature* à WSJF élevé mais à *deadline* serrée.

10.2 Cohérence du Pareto

La sélection *farthest-first* (DEC-007) produit des scénarios visiblement distincts : on peut leur donner des étiquettes parlantes (« rapide », « sûr », « économe »). C'est l'effet recherché, éviter de présenter au RTE cinq variantes du même compromis.

Comment ces étiquettes sont générées. Une fois les cinq scénarios sélectionnés, on regarde sur lequel des six objectifs chacun est le **meilleur du lot** et on en déduit son nom :

Meilleur sur...	Étiquette générée
makespan	« rapide »
score de risque	« sûr »
violation de capacité	« économe »
tardivité WSJF	« prioritaire »
buffer déficit	« avec marge »
surcharge	« équilibré »

Les noms sont volontairement **courts et métier** : le PO et le RTE peuvent les utiliser pour se parler (« je préfère le scénario rapide, mais avec un peu plus de marge ») sans connaître la formule mathématique qui a déterminé le classement. Si plusieurs scénarios sont meilleurs sur le même objectif, on tranche au second meilleur KPI, ce qui produit des étiquettes composées (« rapide-sûr »).

10.3 Reproductibilité

La *seed* fixe, le mode *mock* du LLM et la trace *DecisionTrace* permettent de rejouer une session à l'identique. C'est important ici : sans cela, on ne pourrait ni comparer deux versions du système, ni vérifier un résultat lors de la maintenance.

Une quatrième condition, moins évidente, s'est révélée nécessaire : que les requêtes de chargement imposent un **ordre explicite**. Une graine fixe ne suffit pas si la base renvoie les mêmes lignes dans un ordre différent d'une exécution à l'autre. Deux campagnes lancées avec les mêmes réglages donnaient sinon des chiffres différents.

11 Limites et ce qui n'est pas pris en compte

Le périmètre du prototype a été précisé en section 2.5. Cette section recense ce qui en a été volontairement exclu, les limites algorithmiques connues, et les limites expérimentales du dataset de référence.

11.1 Hors-périmètre fonctionnel

- **Données réelles Jira** non encore ingérées : le passage en production nécessite l'adaptation au format Jira Agile@Scale ainsi que la gestion des autorisations.

- **Multi-ART** non supporté : la planification se fait sur un ART à la fois. Le multi-projet existe (épique → feature) mais pas le multi-ART simultané, traité dans la littérature [16].
- **Replanning temps réel** non traité : le système fonctionne en mode *batch*, le *replanning* continu en cours de PI relève d'une autre classe de problème.
- **Apprentissage par renforcement** non implémenté : l'hybridation NSGA-II + Q-Learning de [4] reste à explorer.

11.2 Limites algorithmiques

- **NSGA-II avec crowding distance** : NSGA-III [9, 11] serait plus adapté pour 6 objectifs.
- **Pas de tampon global de fin de PI**. En l'état, on insère un petit tampon *après chaque story à risque* pour encaisser ses éventuels retards (c'est ce qu'on appelle un *feeding buffer*). La méthode de Goldratt préconise en plus un **gros tampon unique en toute fin de PI** (le *project buffer*), qui sert de filet de sécurité agrégé pour tout l'ART. On ne l'a pas encore mis en place : le RTE pilote donc des dizaines de petits tampons locaux au lieu d'un seul tampon global, ce qui rend plus difficile de voir si le PI dans son ensemble est en danger.
- **span_factor = 2 est une borne basse** : la loi de Kingman donne un lead-time qui croît hyperboliquement au-delà de 80 % d'utilisation [25] ; sur projets à variance forte, le coefficient sous-estime le lead-time réel.
- **Taille des instances non éprouvée**. Les cinq jeux de données comptent une centaine de *stories*, ce qui correspond à un ART. Rien n'a été mesuré au-delà, et la littérature signale précisément que NSGA-II se dégrade quand le nombre de variables de décision devient grand [5]. Un ART deux fois plus gros, ou plusieurs ART traités ensemble, demanderaient de refaire la campagne avant d'annoncer le moindre chiffre.

11.3 Limites expérimentales

Les menaces à la validité de la campagne sont analysées en section 9 et ne sont pas répétées ici. Deux limites propres au dataset de référence méritent d'y être ajoutées.

Sur **seed=42**, le *gap* de capacité reste négatif sur toutes les compétences, donc le scénario HITL du retrait IA en goulot *data* ne se déclenche pas spontanément. Un mode **--amplify-bottleneck** ou un enrichissement des templates avec davantage de *features data* est nécessaire pour rendre la démonstration cliniquement déclenchable. C'est d'autant plus notable que la campagne montre par ailleurs qu'un goulot de compétence **existe bel et bien** sur cette instance et qu'il fixe à lui seul le makespan atteignable : le détecteur mesure la demande agrégée par compétence, alors que la contrainte réelle porte sur la ressource individuelle qui se trouve être la seule éligible.

Enfin, la comparaison porte sur des variantes de NSGA-II et sur des heuristiques gloutonnes, pas sur un solveur exact ni sur les autres métaheuristiques multi-objectifs de la littérature (NSGA-III, SPEA2, MOEA/D). Situer SAFESCALE par rapport à cet état

de l'art demanderait de réimplémenter ces algorithmes sur le même modèle, ce qui dépasse le périmètre de ce mémoire.

12 Points à explorer

Au-delà de la version livrée, six pistes d'extension sont identifiées. Elles sont volontairement techniques, chacune adresse un point précis de l'algorithme ou du système, mais on les annonce ici sans rentrer dans les détails d'implémentation. Le lecteur intéressé par les mécanismes sous-jacents peut se reporter à la section 3 (état de l'art).

12.1 P2 : Passage à NSGA-III pour mieux gérer 6 objectifs

En une phrase : remplacer NSGA-II par une variante plus récente, NSGA-III, mieux outillée pour comparer des plannings sur beaucoup de critères en même temps.

NSGA-II utilise la *crowding distance* pour préserver la diversité du front de Pareto. Cette technique est efficace jusqu'à trois objectifs, mais perd en précision au-delà. NSGA-III remplace ce mécanisme par un système de **points de référence** placés sur un hyperplan normalisé, qui guide explicitement la couverture du front [9, 11, 10]. À six objectifs comme nous en avons, le gain est documenté dans la littérature [6] ; le coût est un paramétrage supplémentaire (placement des points), qui demande sa propre étude.

12.2 P3 : NSGA-II + Q-Learning

En une phrase : laisser le système *apprendre* les bonnes dates de début, au lieu de les choisir par une règle fixe.

[4] démontre qu'apprendre une politique de *timing* (choix du **start_day** de chaque activité) comme processus de décision markovien améliore les résultats par rapport à NSGA-II pur, PSO et ACO. C'est le complément naturel à notre seed *urgency* qui repose actuellement sur une heuristique fixe.

12.3 P4 : Données réelles Jira et multi-ART

En une phrase : brancher SAFESCALE sur les vraies données Jira et lui permettre de planifier plusieurs ART à la fois.

[16] adapte NSGA-II au multi-projet avec critères de durabilité, comblant un vide dans la littérature. Combiné à un adaptateur Jira Agile@Scale, ce travail permettrait le passage du prototype à un cadre industriel.

12.4 P5 : Project buffer global

En une phrase : remplacer les dizaines de petits tampons locaux par un seul gros tampon global de fin de PI, plus facile à piloter.

Consolider les *feeding buffers* locaux de DEC-018 en un *project buffer* global de fin de PI au sens de [24] permettrait au RTE de piloter un seul tampon agrégé plutôt que des dizaines de buffers locaux. Le coût algorithmique est faible mais le changement de modèle mental côté utilisateur demande une étude.

12.5 P6 : Profils de ressource flexibles

En une phrase : laisser le système ajuster en cours de route combien de personnes travaillent sur une story,

plutôt que de le figer à la planification.

[15] traite le cas où la quantité de ressource par activité est déterminée en cours d'exécution plutôt que figée à la planification. Sa contribution utile ici n'est pas l'algorithme mais les schémas de génération de planning qui l'accompagnent : décider non plus seulement de l'ordre des activités, mais aussi de la combinaison de personnes affectée à chacune. Pertinent pour les ressources partagées entre équipes SAFe via les *component teams*.

12.6 P7 : Valoriser les équipes en sous-activité

En une phrase : quand une équipe a de la capacité libre, lui proposer d'anticiper des stories du PI suivant ou de venir en aide à une équipe surchargée.

Le retour terrain fait apparaître un cas symétrique de la surcapacité : des équipes engagées en dessous de leur potentiel sur un PI donné. Deux leviers sont à instrumenter dans l'outil : (i) proposer au RTE d'**anticiper des stories du PI suivant** compatibles avec les compétences disponibles de l'équipe sous-utilisée ; (ii) permettre à une équipe en sous-activité de **prendre en charge des stories d'autres équipes surchargées**, sous réserve de compatibilité de spécialisation. La modélisation des compétences (cf. section 4) et le mécanisme de retrait/ajout au step 3 posent déjà les briques nécessaires ; il reste à câbler les règles de décision et l'interface RTE pour ces deux mouvements.

13 Conclusion

Ce mémoire présente SAFESCALE, un prototype fonctionnel pour la planification SAFe PI Planning combinant NSGA-II à six objectifs, architecture multi-agents orchestrée et marges de sécurité inspirées de la Critical Chain. L'apport principal n'est pas un nouvel algorithme mais l'assemblage d'éléments déjà éprouvés ([3, 1, 22, 25]) dans le contexte précis du PI Planning SAFe, WSJF, sprints, équipes spécialisées, statut de *readiness*, dépendances inter-features, un contexte que l'entretien avec un Product Owner a aidé à cadrer.

Le tableau des décisions (table 2) relie chaque choix de conception à sa source, un article ou un point relevé lors de l'entretien PO. C'est un moyen, plus qu'un effet de style, de pouvoir justifier le système devant un jury.

À ce stade, le travail reste un prototype, mais un prototype mesuré : le flow tourne de bout en bout sur le dataset de référence et la campagne expérimentale (section 9) établit trois choses. Les plannings produits sont faisables ; l'optimisation multi-objectifs divise par six la violation de capacité par rapport à un ordonnanceur glouton, sans pouvoir améliorer un makespan borné par un goulot de compétence ; et le seuil bloquant de l'étape 3 est ce qui empêche le système de produire un engagement structurellement intenable. Elle montre aussi ses propres limites : sur les deux améliorations apportées à NSGA-II, une seule est démontrée, et la dominance de Pareto sature à six objectifs. Les retours qualitatifs suggèrent par ailleurs que le découpage en cinq étapes avec HITL conditionnel rend les arbitrages lisibles ; cela, en revanche, reste à confirmer par une évaluation avec des utilisateurs. Les limites (section 11) et les pistes d'extension (section 12) indiquent les prochaines étapes vers un usage réel.

A Annexe : Glossaire des sigles

ACO

Ant Colony Optimization, optimisation par colonies de fourmis.

ART

Agile Release Train, regroupement de 5 à 15 équipes qui livrent ensemble.

BDI

Beliefs, Desires, Intentions, modèle d'agent autonome (non retenu ici).

CNP

Capacity Negotiation Protocol, étape 3 du flow, négociation de capacité.

CPO

Chief Product Owner, priorise les features de son périmètre.

DAG

Directed Acyclic Graph, graphe orienté sans cycle, modélise les dépendances.

EDM

Engineering Delivery Manager, responsable de la livraison côté ingénierie.

HITL

Human-In-The-Loop, intervention d'un humain dans la boucle de décision.

LLM

Large Language Model, grand modèle de langage, utilisé ici comme traducteur.

MAS

Multi-Agent System, système multi-agents.

ML

Machine Learning, apprentissage automatique.

MOEA

Multi-Objective Evolutionary Algorithm, algorithme évolutif multi-objectifs.

MS-RCMPSP

Multi-Skill Resource-Constrained Multi-Project Scheduling Problem, extension multi-compétences et multi-projets du RCMPSP.

NSGA-II/III

Non-dominated Sorting Genetic Algorithm, algorithme génétique de tri non-dominé, versions 2 et 3.

PI

Program Increment, cycle de livraison de 8 à 12 semaines.

PO

Product Owner, porte le contenu d'une équipe.

PSO

Particle Swarm Optimization, optimisation par essaim particulaire.

RCPSP

Resource-Constrained Project Scheduling Problem, problème d'ordonnement de projet sous contraintes de ressources.

ROAM

Resolved, Owned, Accepted, Mitigated, tableau SAFe de gestion des risques.

RTE

Release Train Engineer, facilitateur de l'ART.

SAFe

Scaled Agile Framework, cadre d'agilité à l'échelle.

SM

Scrum Master, facilitateur de l'équipe.

SP

Story Points, unité d'estimation relative de l'effort.

WSJF

Weighted Shortest Job First, score de priorisation : valeur sur effort.

B Annexe : Glossaire des termes techniques

Optimisation et algorithmes évolutifs

Algorithme évolutif

Algorithme qui imite l'évolution naturelle (croisement, mutation, sélection) sur une population de solutions.

Algorithme génétique

Algorithme évolutif qui fait évoluer une population par croisement et mutation, à la manière de la sélection naturelle.

Métaheuristique

Méthode générale qui trouve de très bonnes solutions sans garantir l'optimum exact.

Individu / chromosome

Une solution candidate, ici, un planning complet.

Gène

Plus petit élément d'un chromosome ; ici, le couple (*story*, ressource).

Croisement

Combinaison de deux solutions parents pour produire un enfant.

Mutation

Modification aléatoire d'une partie d'une solution.

Élitisme

Conservation systématique des meilleures solutions d'une génération à l'autre.

Front de Pareto

Ensemble des meilleurs compromis : aucune solution du front ne peut être améliorée sur un critère sans en dégrader un autre.

Dominance

A domine B si A est au moins aussi bonne partout et strictement meilleure quelque part.

Crowding distance

Mesure favorisant des solutions bien réparties sur le front.

Recherche locale

Amélioration pas à pas d'une solution par petites modifications.

Algorithme mémétique

Algorithme évolutif augmenté d'une recherche locale.

Seed (initialisation)

Solution de départ intelligente injectée dans la population initiale.

Seed (graine aléatoire)

Nombre fixant le hasard ; à seed fixe, le programme est reproductible.

Distance de Hamming

Nombre de positions où deux solutions diffèrent.

NP-difficile

Problème dont aucun algorithme connu ne donne l'optimum en temps raisonnable à grande taille.

Ordonnancement**Lead-time**

Temps écoulé entre le début et la fin d'une tâche, attentes comprises, par opposition au temps de travail effectif.

Makespan

Durée totale du projet, du début de la première story à la fin de la dernière.

Chemin critique

Plus longue chaîne de dépendances ; fixe la durée minimale du projet.

Tri topologique

Mise en ordre d'un DAG respectant toutes les dépendances.

Décodage

Transformation d'un chromosome en planning concret (dates de début/fin).

Tardivité pondérée

Retard d'une livraison multiplié par l'importance de l'élément en retard.

Goulot d'étranglement

Compétence trop demandée par rapport au nombre de personnes qui la maîtrisent.

SAFe et agilité**Epic / Feature / Story**

Niveaux de découpage du travail ; la *story* est l'unité de planification.

Story point

Unité d'estimation relative de l'effort d'une story.

Vélocité

Story points réalisés par équipe et par sprint.

Poker planning

Estimation collective de l'effort des stories.

Readiness

Niveau de préparation d'une feature avant planification.

Sprint / cadence

Itération courte (3 semaines) ; la cadence est le rythme régulier des cérémonies.

Component team

Équipe transverse ; exception au principe « une équipe = une feature ».

Architecture et modélisation**Orchestration / émergence**

Orchestration : un chef d'orchestre central pilote les agents. Émergence : agents autonomes sans chef.

HITL conditionnel

L'humain n'est sollicité que si un problème est détecté.

Mock

Fausse version d'un composant pour des tests reproductibles.

Durée-effort / durée-calendaire

L'effort réel (jours-personne) contre le temps réellement écoulé (lead-time).

Span factor

Coefficient transformant la durée-effort en durée-calendaire (pivot 2.0).

Loi de Kingman

Au-delà de ~80 % d'utilisation, les délais d'attente explosent, justifie le buffer 20 %.

Critical Chain

Méthode de Goldratt : regrouper les marges en buffers ciblés plutôt que gonfler chaque estimation.

Feeding buffer

Tampon de temps inséré après une story risquée.

Score de risque déterministe

Score 0-1 calculé par une formule fixe (mêmes entrées $\Rightarrow$ même score).

Évaluation**Dataset synthétique**

Jeu de données généré artificiellement, aux dimensions calées sur un cas réel.

Reproductibilité

Propriété d'une expérience qui redonne le même résultat à chaque exécution.

Hypervolume

Volume de l'espace des objectifs couvert par un front de Pareto ; plus grand = meilleur. N'a de sens que rapporté à un repère fixe, commun aux fronts comparés.

Spread

Mesure de la bonne répartition des solutions sur le front.

IGD+

Distance moyenne d'un front de référence au front évalué, en ne comptant que les écarts défavorables ; plus bas = meilleur.

C-metric

Part des solutions d'un front couvertes par au moins une solution d'un autre front. Indicateur non symétrique.

Ablation

Protocole de comparaison consistant à désactiver une contribution à la fois, pour mesurer ce que chacune apporte réellement.

Taille d'effet

Mesure de l'ampleur d'un écart, indépendante du nombre d'observations ; ici A_{12} de Vargha-Delaney.

Borne inférieure

Valeur qu'aucune solution ne peut franchir, calculée analytiquement ; sert à situer un résultat quand l'optimum est inconnu.

Score composite

Note unique combinant les six objectifs normalisés et pondérés.

Dettes techniques

Travail de correction connu, reporté volontairement et documenté.

Références

- [1] X. Wen, Q. Song, Y. Qian *et al.*, « Effective Improved NSGA-II Algorithm for Multi-Objective IPPS », *Mathematics*, MDPI, 11(16) :3523, 2023.
<https://ideas.repec.org/a/gam/jmathe/v11y2023i16p3523-d1217570.html>
- [2] J. Wang, L. Xu, S. Sun *et al.*, « Multi-objective optimization using improved NSGA-II for IPPS in a machining job shop for large-size valve », *PLoS ONE*, 19(6) :e0306024, 2024.
<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0306024>

- [3] S. C. Burmeister, D. Guericke, G. Schryen, « A memetic NSGA-II for the multi-objective flexible job shop scheduling problem with real-time energy tariffs », *Flexible Services and Manufacturing Journal*, Springer, 36(4) :1530-1570, 2024.
<https://link.springer.com/article/10.1007/s10696-023-09517-7>
- [4] H. Yang, Z. Wang, Y. Gao, W. Zhou, « Bi-objective multi-mode resource-constrained multi-project scheduling using combined NSGA-II and Q-learning », *Applied Soft Computing*, 152 :111201, 2024.
<https://www.sciencedirect.com/science/article/abs/pii/S156849462301219X>
- [5] A. J. Nebro, J. Galeano-Brajones, F. Luna, C. A. Coello Coello, « Is NSGA-II Ready for Large-Scale Multi-Objective Optimization ? », *Mathematical and Computational Applications*, MDPI, 27(6) :103, 2022.
<https://www.mdpi.com/2297-8747/27/6/103>
- [6] Z. Zhang, « Multi-objective optimization method for building energy-efficient design based on multi-agent-assisted NSGA-II », *Energy Informatics*, Springer, 7(1), 2024.
<https://link.springer.com/article/10.1186/s42162-024-00394-4>
- [7] S. Verma, M. Pant, « A Comprehensive Review on NSGA-II for Multi-Objective Combinatorial Optimization Problems », IEEE Xplore, 2021.
<https://ieeexplore.ieee.org/document/9393947/>
- [8] Wang Yahui, Shi Ling, Zhang Cai *et al.*, « NSGA-II algorithm and application for multi-objective flexible workshop scheduling », *Sage Journals*, 2020.
<https://journals.sagepub.com/doi/full/10.1177/1748302620942467>
- [9] K. Deb, H. Jain, « An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Non-dominated Sorting Approach, Part I », *IEEE TEVC*, 18(4) :577-601, 2014.
<https://ieeexplore.ieee.org/document/6600851>
- [10] Z. Ling, Y. Chen, « Research on Multi-Objective Optimization of Project Scheduling based on NSGA-III : Trade-off among Project Duration, Resource Allocation, and Robustness », *International Core Journal of Engineering*, 11(4) :413-422, 2025.
<https://bcpublication.org/index.php/ICJE/article/view/8360>
- [11] V. H. S. Pham, N. T. P. Hoang, « Advancing multi-objective optimization in resource management using the NSGA-III algorithm », *Canadian Journal of Civil Engineering*, 52(7) : 1319-1335, 2025.
<https://cdnsiencepub.com/doi/10.1139/cjce-2024-0298>
- [12] V. Valls, F. Ballestín, S. Quintanilla, « A hybrid genetic algorithm for the resource-constrained project scheduling problem », *European Journal of Operational Research*, 185(2) :495-508, 2008.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221707000616>
- [13] E. Goncharov, « A hybrid heuristic algorithm for the resource-constrained project scheduling problem », arXiv :2502.18330, 2025.
<https://arxiv.org/abs/2502.18330>
- [14] R. Torba, S. Dauzère-Pérès, C. Yugma, C. Gallais, J. Pouzet, « Solving a real-life multi-skill resource-constrained multi-project scheduling problem », *Annals of Operations Research*, Springer, 338(1) :69-114, 2024.
<https://link.springer.com/article/10.1007/s10479-023-05784-7>
- [15] X. Luo, S. Guo, B. Du, X. Luo, J. Guo, « Multi-Objective Multi-Skill Resource-Constrained Project Scheduling Considering Flexible Resource Profiles », *MDPI Applied Sciences*, 14(5) :1921, 2024.
<https://www.mdpi.com/2076-3417/14/5/1921>
- [16] M. A. Lostumbo, M. Bara Iniesta, L. Calvet, « Sustainable Optimization of Construction Project Portfolio Management Through a Data-Driven Approach », dans *Decision Sciences*, Lecture Notes in Computer Science, Springer, 2025, p. 60-71.
https://doi.org/10.1007/978-3-031-78238-1_6
- [17] H. Yang, Y. Yuan, S. Ye, L. Lin, « Fuzzy Resource Constrained Project Scheduling Optimization with Hybrid Multiobjective Genetic Algorithm », *ACM ICBIC*, p. 280-285, 2021.
<https://dl.acm.org/doi/10.1145/3448748.3448793>
- [18] S. Rodríguez-Ballesteros, J. Alcaraz, L. Anton-Sanchez, « Multi-mode RCPSP with time-dependent resource costs and capacities : A bi-objective approach », *Expert Systems with Applications*, 306 :130955, 2026.
<https://www.sciencedirect.com/science/article/pii/S0957417425045701>
- [19] R. Pellerin *et al.*, « A survey of hybrid metaheuristics for the resource-constrained project scheduling problem », *EJOR*, 280(2), 2020.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221719300475>
- [20] W. Herroelen, B. De Reyck, E. Demeulemeester, « Resource-constrained project scheduling : A survey of recent developments », *Computers & Operations Research*, 25(4), 1998.
<https://www.sciencedirect.com/science/article/abs/pii/S0305054897000551>
- [21] Scaled Agile Inc., *SAFe 6.0 Framework, PI Planning*.
<https://framework.scaledagile.com/pi-planning>
- [22] W. Zhang, G. Xiao, M. Gen, H. Geng, X. Wang, M. Deng, G. Zhang, « Enhancing multi-objective evolutionary algorithms with machine learning for scheduling problems : recent advances and survey », *Frontiers in Industrial Engineering*, 2, 2024.
<https://www.frontiersin.org/journals/industrial-engineering/articles/10.3389/fieng.2024.1337174/full>
- [23] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, « A Fast and Elitist Multiobjective Genetic Algorithm : NSGA-II », *IEEE Transactions on Evolutionary Computation*, 6(2) :182-197, 2002.
<https://ieeexplore.ieee.org/document/996017>
- [24] E. M. Goldratt, *Critical Chain*, North River Press, Great Barrington (MA), 1997. ISBN 978-0-88427-153-6.
- [25] D. G. Reinertsen, *The Principles of Product Development Flow : Second Generation Lean Product Development*, Celeritas Publishing, Redondo Beach (CA), 2009. ISBN 978-1-935401-00-1.
- [26] J. Blazewicz, J. K. Lenstra, A. H. G. Rinnooy Kan, « Scheduling subject to resource constraints : classification and complexity », *Discrete Applied Mathematics*, 5 :11-24, 1983.
<https://www.sciencedirect.com/science/article/pii/0166218X83900124>
- [27] E. Zitzler, L. Thiele, « Multiobjective Evolutionary Algorithms : A Comparative Case Study and the Strength Pareto Approach », *IEEE Transactions on Evolutionary Computation*, 3(4) :257-271, 1999.
<https://ieeexplore.ieee.org/document/797969>
- [28] H. Ishibuchi, H. Masuda, Y. Tanigaki, Y. Nojima, « Modified Distance Calculation in Generational Distance and Inverted Generational Distance », *Evolutionary Multi-Criterion Optimization (EMO 2015)*, LNCS 9019 :110-125, 2015.
https://link.springer.com/chapter/10.1007/978-3-319-15892-1_8
- [29] A. Vargha, H. D. Delaney, « A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong », *Journal of Educational and Behavioral Statistics*, 25(2) :101-132, 2000.
<https://journals.sagepub.com/doi/10.3102/10769986025002101>
- [30] A. Arcuri, L. Briand, « A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering », *33rd International Conference on Software Engineering (ICSE)*, 1-10, 2011.
<https://ieeexplore.ieee.org/document/6032496>